Chapter 3: The Link Layer

Our goals:

- understand principles behind data link layer services:
  - framing
  - error detection, correction
  - reliable data transfer, flow control
  - sharing a broadcast channel: multiple access
  - link layer addressing

Overview:

- link layer services
- framing
- error detection, correction
- reliable data transfer
- multiple access protocols and LANs
- link layer addressing

第三章：数据链路层

我们的目标：

- 理解数据链路层服务背后的原理：
  - 帧编码
  - 错误检测、纠正
  - 可靠数据传输、流量控制
  - 共享广播通道：多路访问
  - 链路层寻址

概述：

- 链路层服务
- 帧编码
- 错误检测、纠正
- 可靠数据传输
- 多路访问协议和局域网
- 链路层寻址

Keypoints and Difficulties

Keypoints:

- Four framing methods
- CRC
- reliable data transfer
- multiple access protocols
- link layer addressing

Difficulties:

- CRC
- Flow control
- Sliding-window protocol
- CSMA/CD

关键点和难点

关键点：

- 四种帧定界方法
- CRC
- 可靠数据传输
- 多路访问协议
- 链路层寻址

困难：

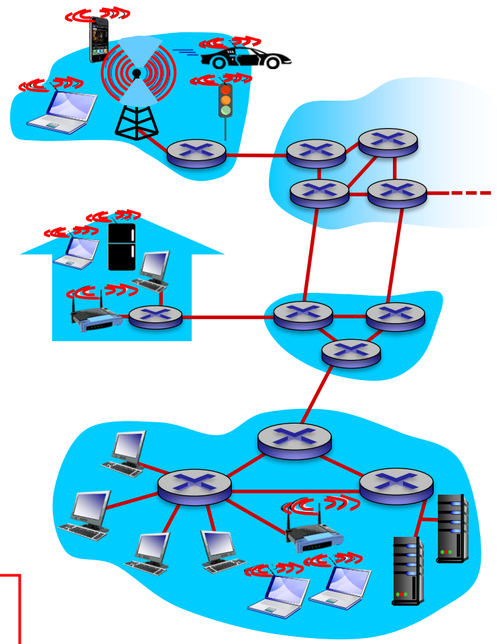
- CRC
- 流量控制
- 滑动窗口协议
- CSMA/CD

Link layer: Introduction

terminology:

- hosts and routers: nodes
- communication channels that connect adjacent nodes along communication path: links
  - wired links
  - wireless links
  - LANs
- layer-2 packet: frame, encapsulates datagram

data-link layer has responsibility of transferring datagram from one node to *physically adjacent* node over a link

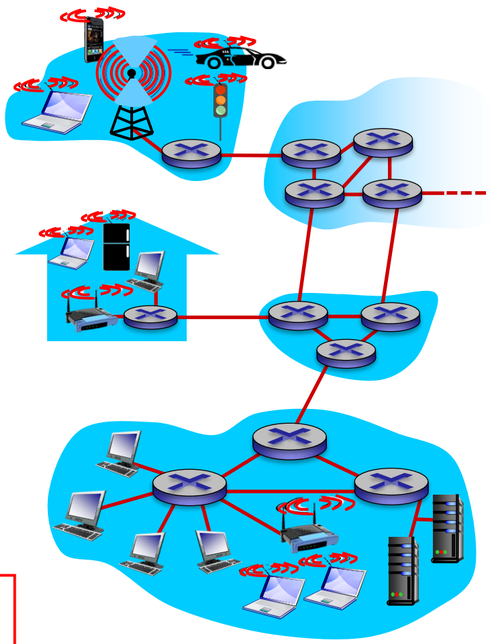


链路层：简介

术语：

- 主机和路由器：节点
- 连接通信路径上相邻节点的通信通道：链路
  - 有线链路
  - 无线链路
  - 局域网
- 第二层数据包：帧封装数据报

数据链路层负责将数据报从一个节点传输到物理相邻节点，通过链路



Link layer: context

- datagram transferred by different link protocols over different links:
  - e.g., Ethernet on first link, frame relay on intermediate links, 802.11 on last link
- each link protocol provides different services
  - e.g., may or may not provide rdt over link

transportation analogy:

- trip from HUST to Shenyang
  - car: HUST to Tianhe Airport
  - plane: Wuhan to Beijing
  - train: Beijing to Shenyang
- tourist = datagram
- transport segment = communication link
- transportation mode = link layer protocol
- travel agent = routing algorithm

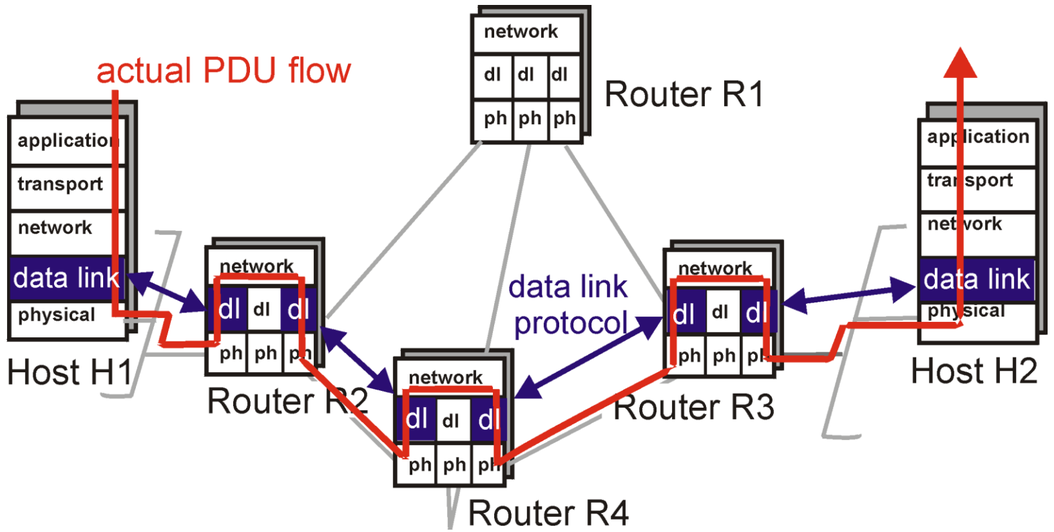
链路层：上下文

- 不同链路协议在不同链路上传输的数据报：
  - 例如，第一跳是以太网，中间链路是帧中继，最后一跳是802.11
- 每个链路协议提供不同的服务
  - 例如，可能或不可能在链路层提供可靠数据传输协议

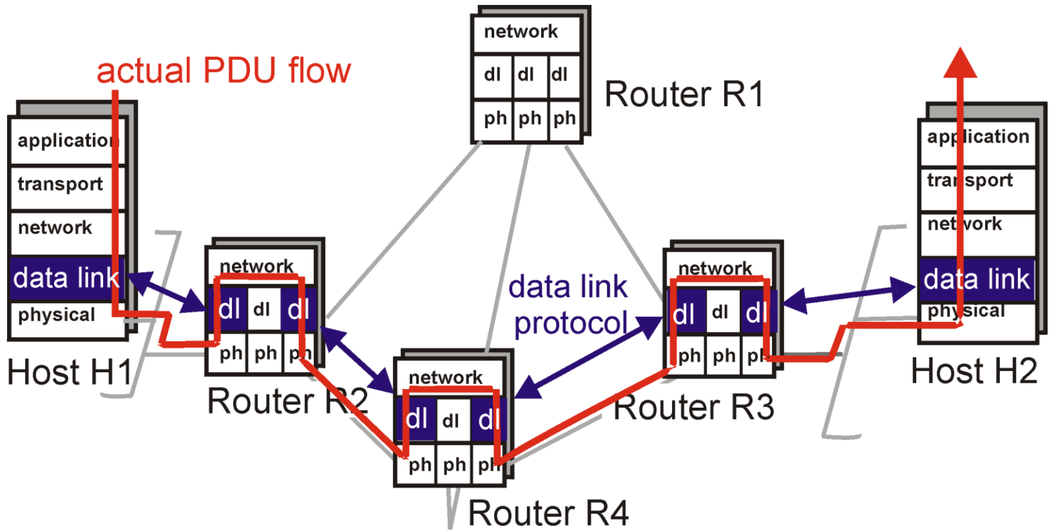
交通类比：

- 从华中科技大学到沈阳的行程
  - 汽车：从华中科技大学到天河机场
  - 飞机：从武汉到北京
  - 火车：从北京到沈阳
- 游客 = 数据报
- 传输段 = 通信链路
- 运输方式 = 链路层协议
- 旅行社 = 路由算法

Link Layer: setting the context

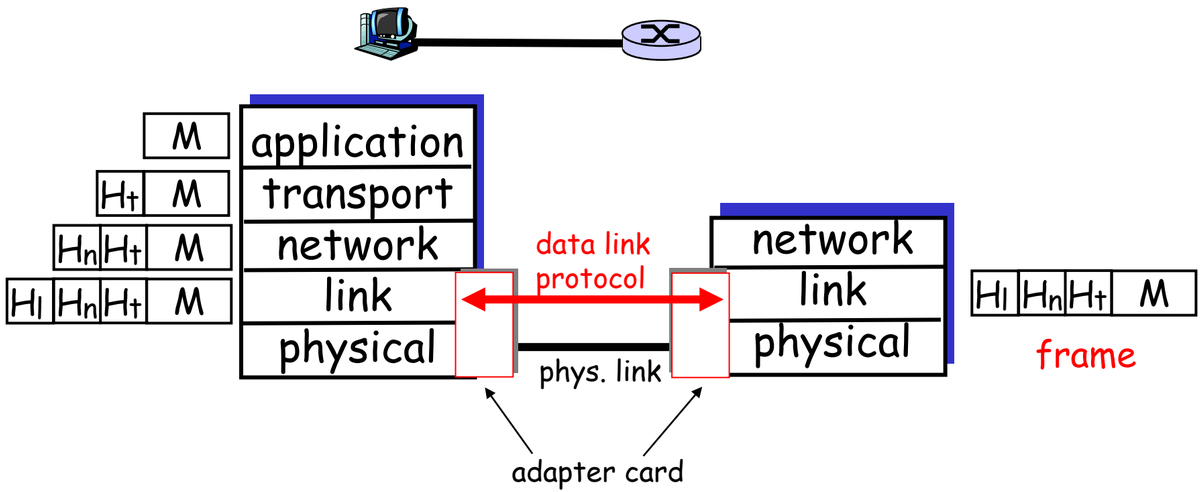


链路层：设定上下文



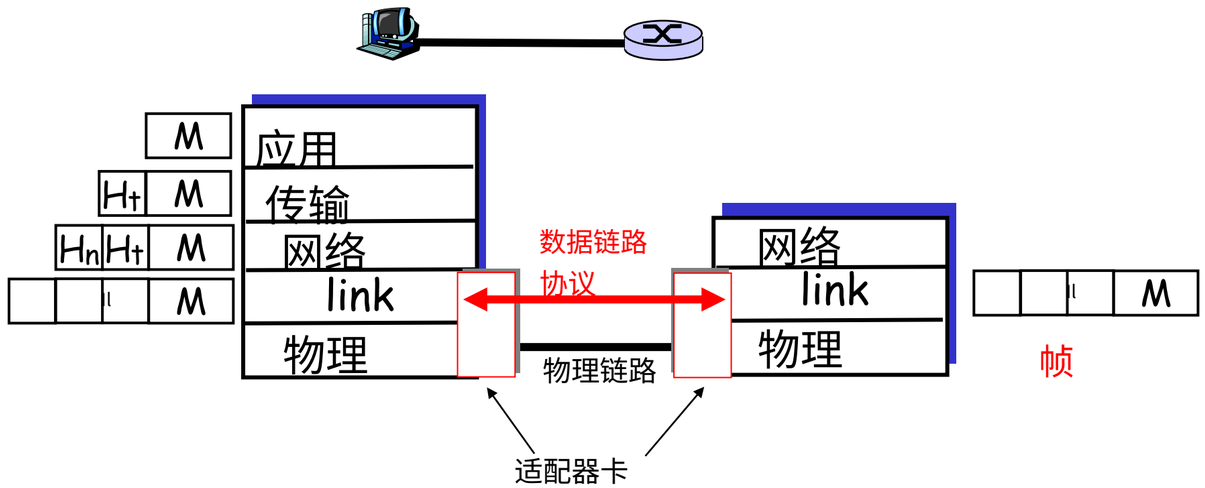
Link Layer: setting the context

- two *physically connected* devices:
  - host-router, router-router, host-host
- unit of data: *frame*



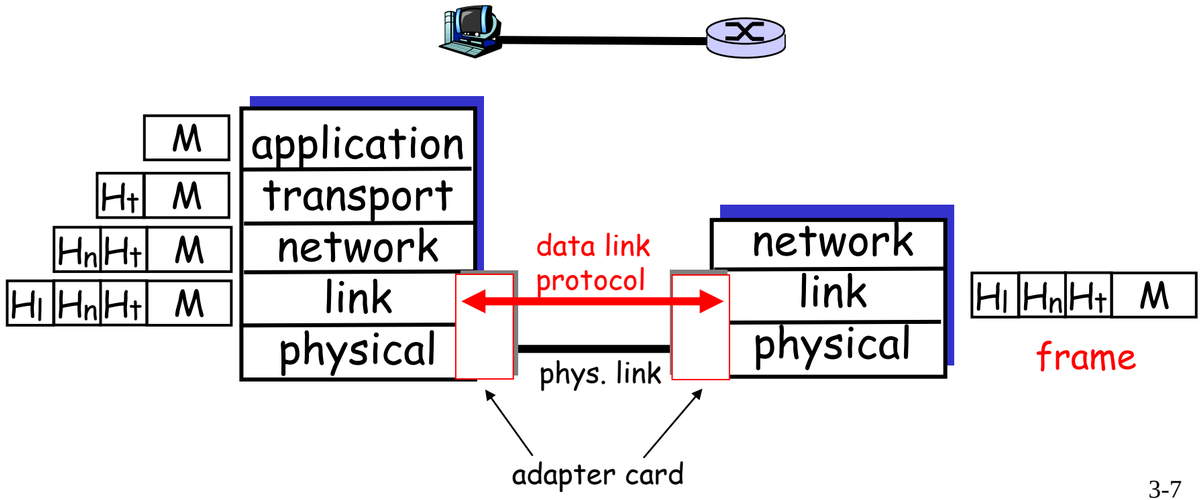
链路层：设定上下文

- 两个物理连接的设备：
  - 主机-路由器，路由器-路由器，主机-主机
- 数据单元： 帧



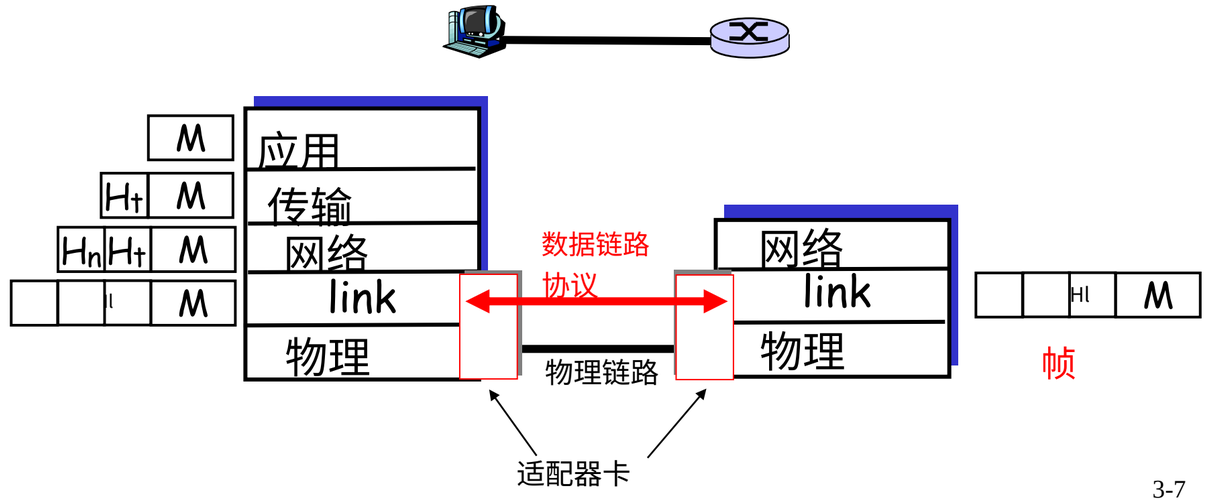
Link Layer: Implementation

- implemented in "adapter"
  - e.g., PCMCIA card, Ethernet card
  - typically includes: RAM, DSP chips, host bus interface, and link interface



链路层：实现

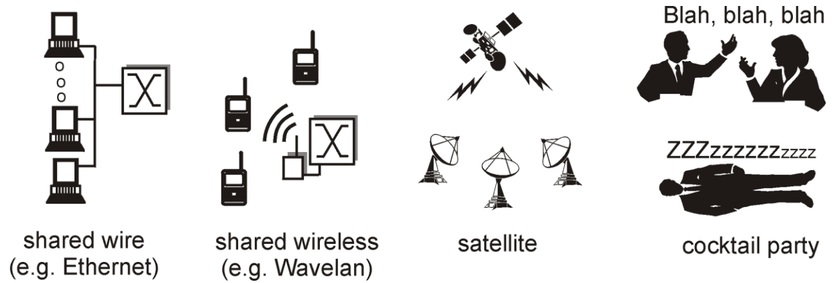
- 在“适配器”中实现
  - 例如，PCMCIA卡、以太网卡
  - 通常包括：RAM、DSP芯片、主机总线接口和链路接口



Multiple Access Links and Protocols

Three types of "links":

- point-to-point (single wire, e.g. PPP, SLIP)
- broadcast (shared wire or medium; e.g, Ethernet, Wavelan, etc.)

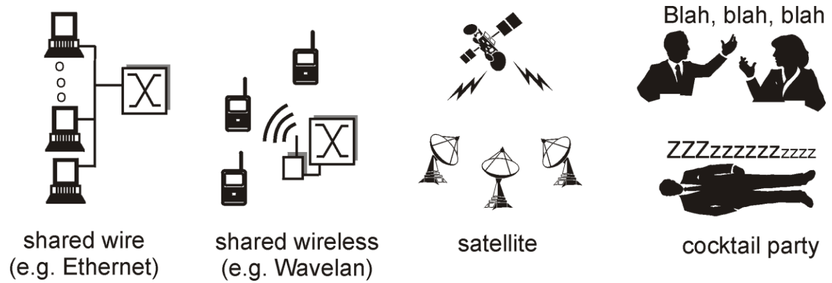


- switched (e.g., switched Ethernet, ATM etc)

多点接入链路与协议

三种"链路"类型：

- 点对点(单线，例如 PPP、SLIP)
- 广播（共享线缆或介质；例如，以太网、Wavelan 等）



- 交换式（例如，交换式以太网、ATM 等）

Link Layer Services

- Framing, link access:
  - encapsulate datagram into frame, adding header, trailer
  - implement channel access if shared medium,
  - 'physical addresses' used in frame headers to identify source, dest
- Reliable delivery between two physically connected devices:
  - Reliable data transfer protocol
  - seldom used on low bit error link (fiber, some twisted pair)
  - wireless links: high error rates

链路层服务

- 帧编码，链路接入：
  - 将数据报封装到帧中，添加头部和尾部 如果共享介质，则实现通道访问 使用帧头中的“物理地址”来标识源地址和目标地址
- 在两个物理连接的设备之间进行可靠传输：
  - 可靠数据传输协议 很少用于低比特错误链路（光纤、某些双绞线） 无线链路：高错误率

Link Layer Services (more)

- Flow Control:
  - pacing between sender and receivers
- *Error Detection*:
  - errors caused by signal attenuation, noise.
  - receiver detects presence of errors:
    - signals sender for retransmission or drops frame
- Error Correction:
  - receiver identifies *and corrects* bit error(s) without resorting to retransmission

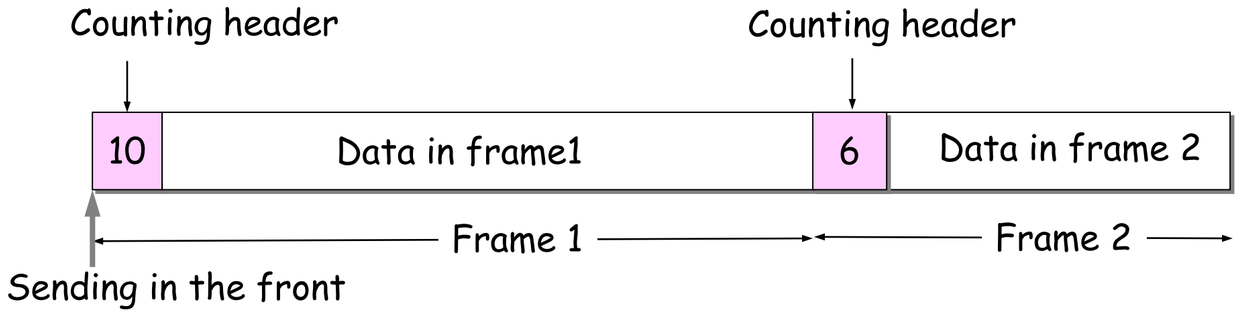
链路层服务（更多）

- 流量控制：
  - 发送者与接收者之间的时序间隔
- 错误检测：
  - 由信号衰减、噪声引起的错误
  - 接收者检测到错误的存在：
    - 信号发送者请求重传或丢弃帧
- 纠错：
  - 接收者识别并纠正位错误，无需重传

3-10

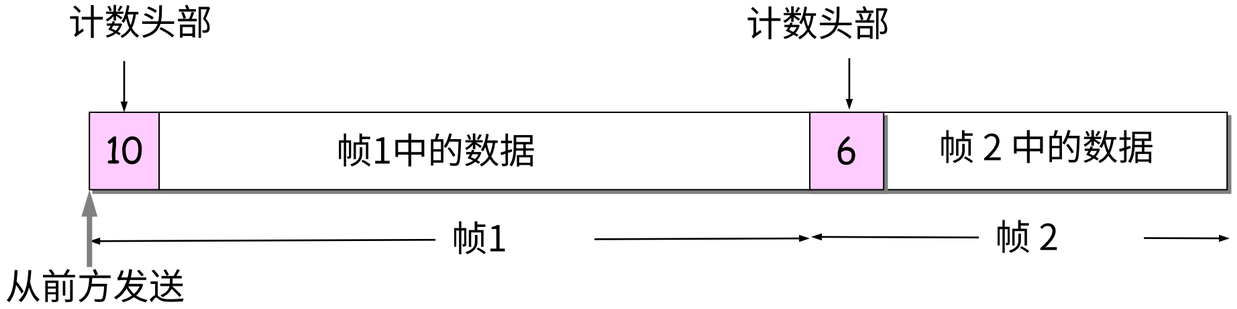
Framing

- Character count method :
  - encapsulate datagram into frame, adding header (character number)



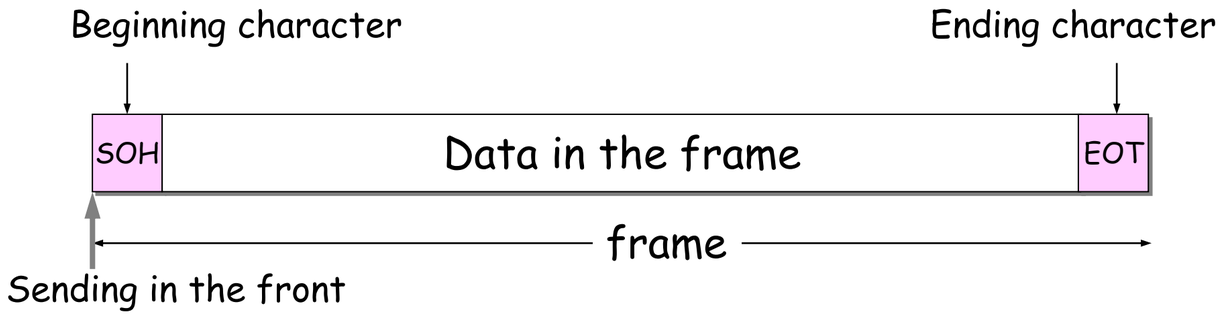
帧编码

- 字符计数方法：
  - 将数据报封装到帧中，添加头部（字符数量）



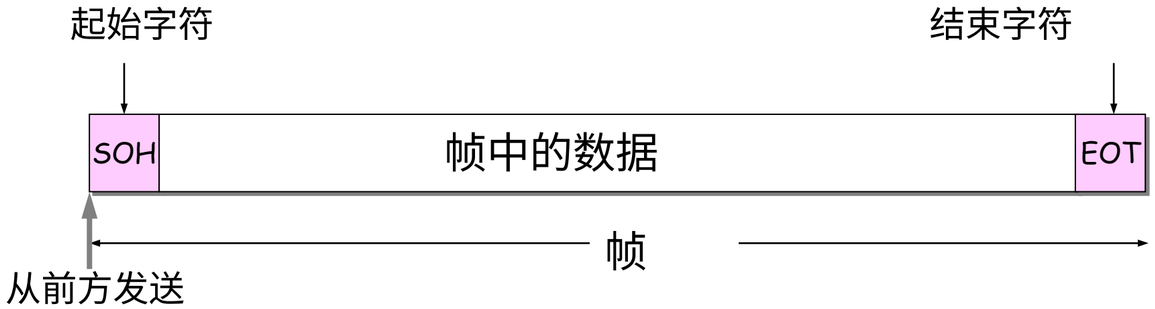
Framing

- First and tail bound method based on character:
  - encapsulate datagram into frame, adding header, trailer (character)



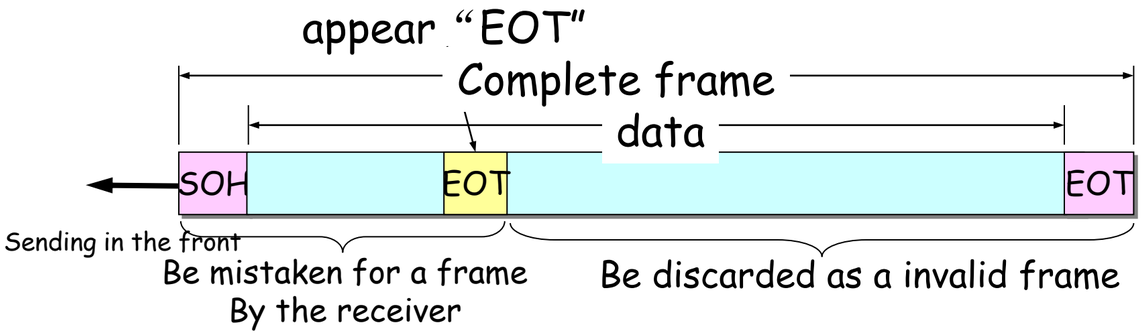
帧编码

- 基于字符的首尾界限法：
  - 将数据报封装入帧，添加头部、尾部（字符）



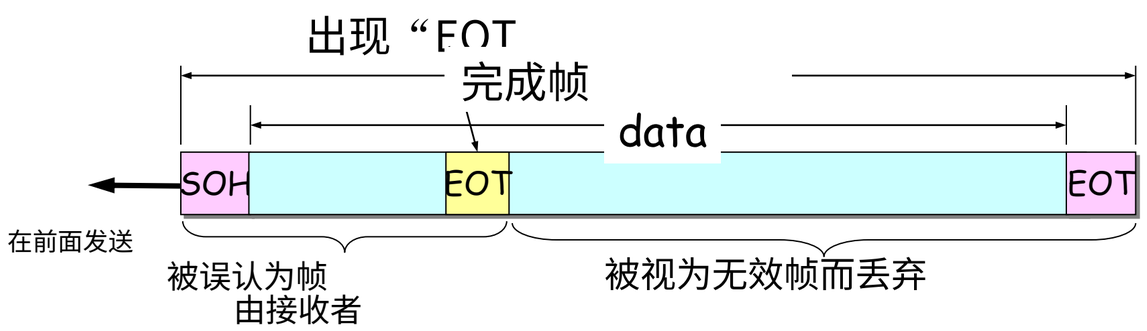
Framing

- First and tail bound method based on character:
  - encapsulate datagram into frame, adding header, trailer (character)



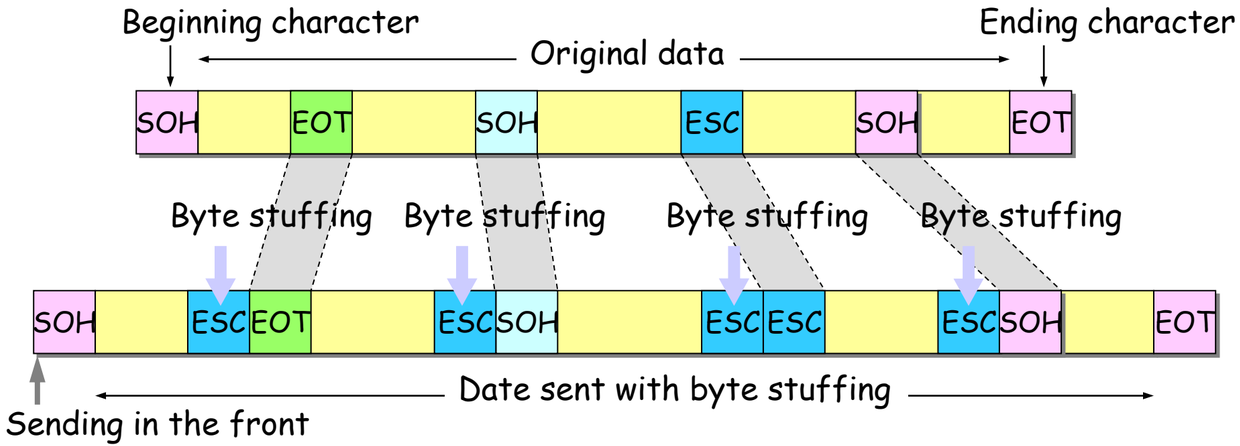
帧编码

- 基于字符的首尾界限法：
  - 将数据报封装到帧中，添加头部、尾部（字符）



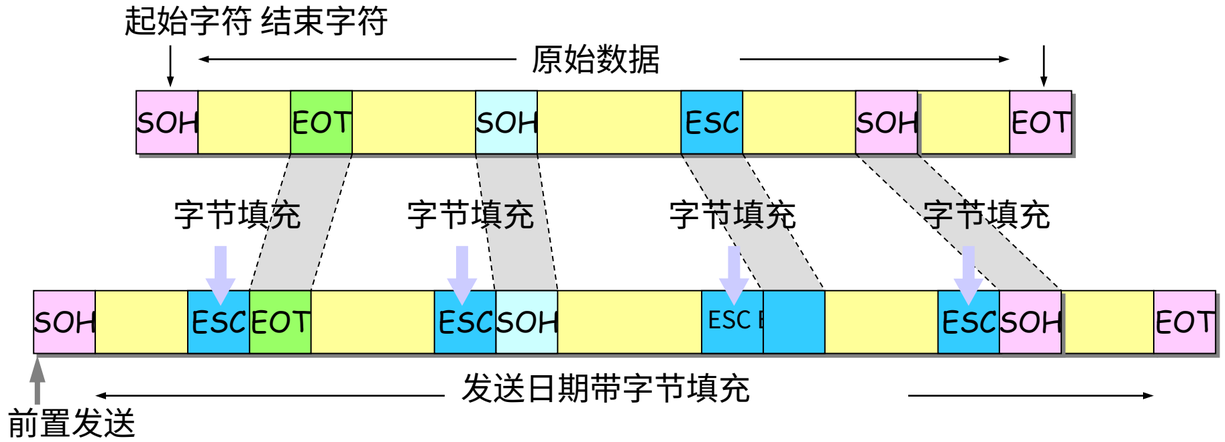
Framing

- First and tail bound method based on character:
  - Inserting/ stuffing a Escape character before the special character in the data part



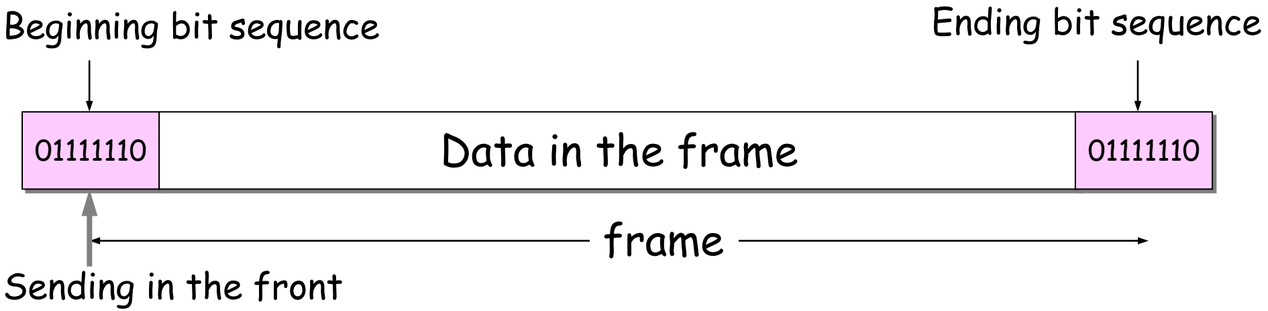
帧编码

- 基于字符的首尾界限法：
  - 在数据部分的特殊字符前插入/填充转义字符



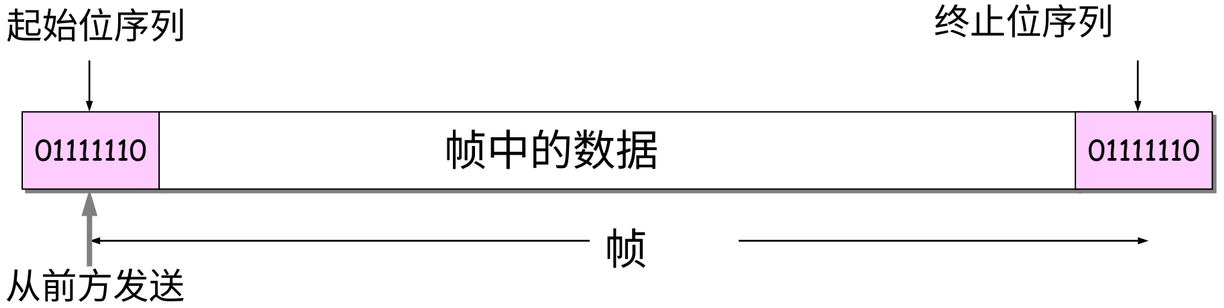
Framing

- First and tail bound method based on bit:
  - encapsulate datagram into frame, adding header, trailer (bit sequence)



帧编码

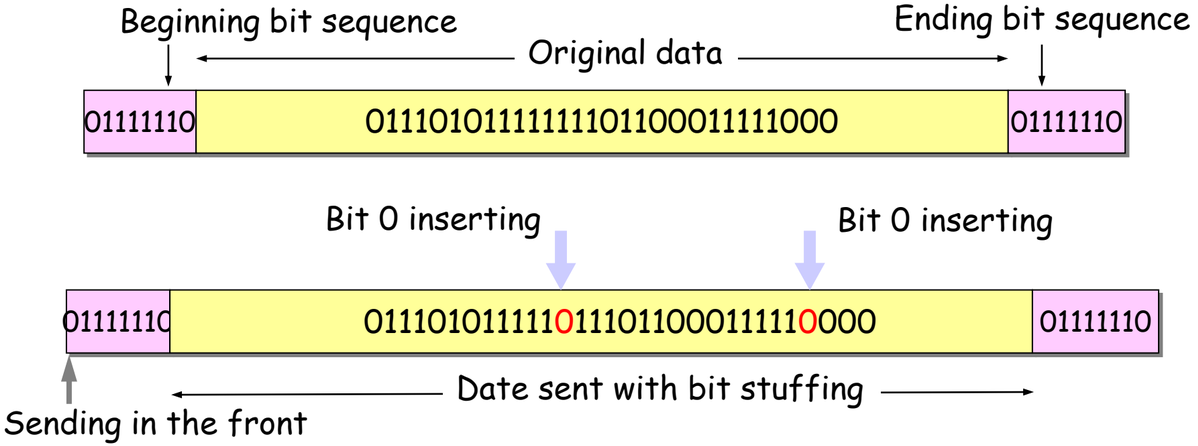
- 基于位的首尾界限法：
  - 将数据报封装到帧中，添加头部、尾部（位序列）



3-15

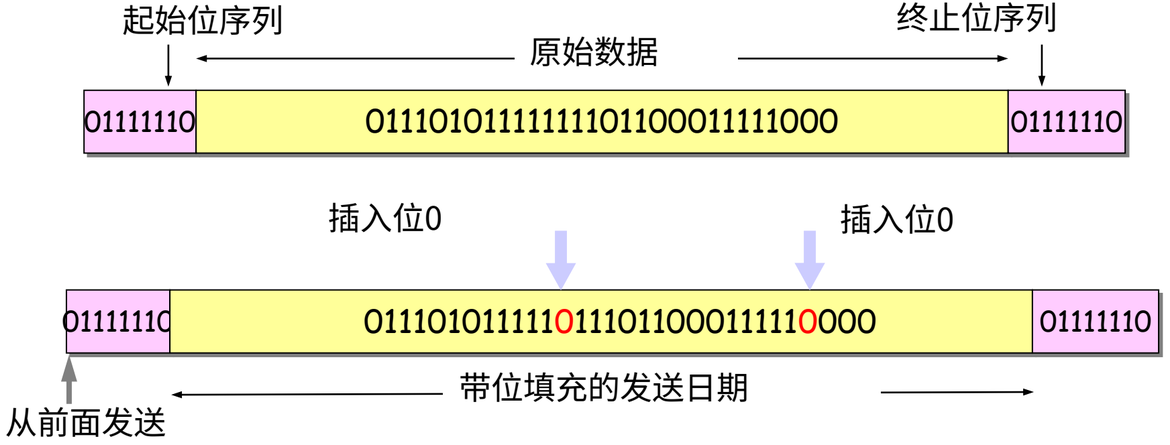
Framing

- First and tail bound method based on bit:
  - Inserting a bit "0" after successive five bits "0" in the transmitter; vice versa



帧编码

- 基于位的首尾界限法：
  - 在连续的 五个位 "0" 之后插入位 "0" ；反之亦然



Framing

- Physical layer coding violation method:
  - encapsulate datagram into frame without stuffing
  - Only be used in the networks with redundancy coding technology in the physical layer
- For example (IEEE 802.11 with Manchester code):
  - Bit 1 with level jump mode from high to low
  - Bit 0 with level jump mode from low to high
  - Level jump modes from high to high or from low to low can be used for beginning and ending of a frame

帧编码

- 物理层编码违规方法：
  - 将数据报封装到帧中 不进行填充 仅用于物理层具有冗余编码技术的网络
- 例如（使用曼彻斯特编码的 IEEE 802.11）：
  - 位1 在电平跳变模式下从高电平跳变到低电平 位0在电平跳变模式下从低电平跳变到高电平 电平跳变模式可以从高电平跳变到高电平或从低电平跳变到低电平用于帧的开始和结束

Exercise-1

1. HDLC protocol sets the bit stream 0111110001111110 after framing as ( )

A. 011111000011111010

B.01111100011111010111110

C.01111100011111010

D.01111100011111100111101

练习-1

1. HDLC协议在帧编码后设置比特流0111110001111110为（）

A. 011111000011111010

B.01111100011111010111110

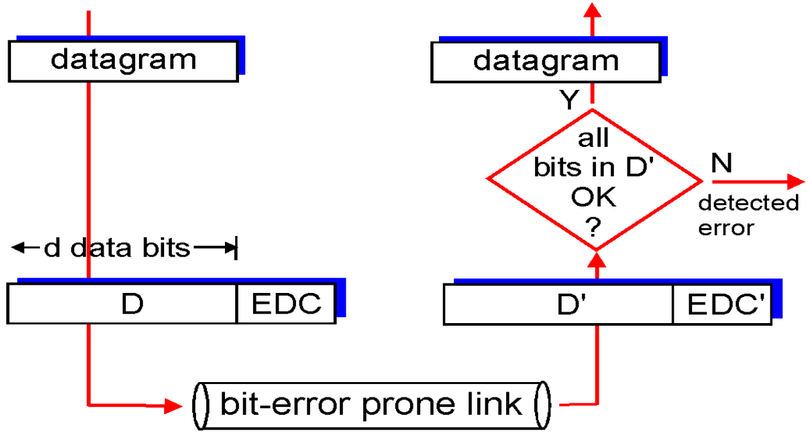
C.01111100011111010

D.01111100011111100111101

Error Detection

EDC= Error Detection and Correction bits (redundancy)
D = Data protected by error checking, may include header fields

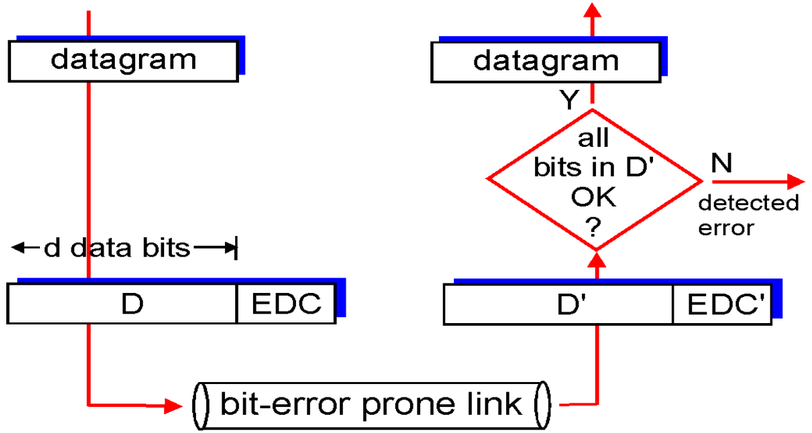
- Error detection not 100% reliable!
  - protocol may miss some errors, but rarely
  - larger EDC field yields better detection and correction



错误检测

EDC= 错误检测与纠正位（冗余） D = 受错误校验保护的数据，可能包含头部字段

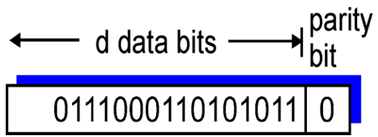
- 错误检测并非100%可靠！
  - 协议可能会漏检一些错误，但很少情况下• 更大的EDC字段能带来更好的检测和纠正效果



Parity Checking

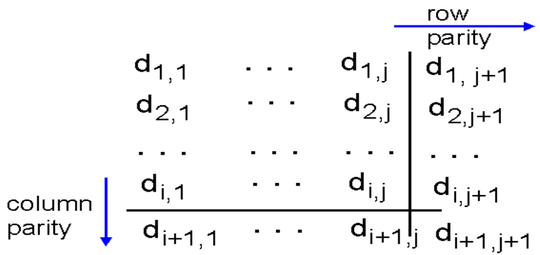
Single Bit Parity:

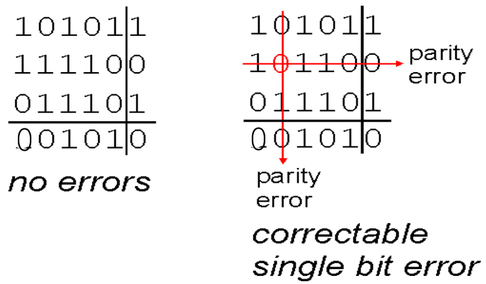
Detect single bit errors



Two Dimensional Bit Parity:

Detect *and correct* single bit errors

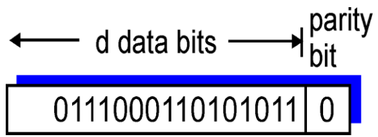




奇偶校验

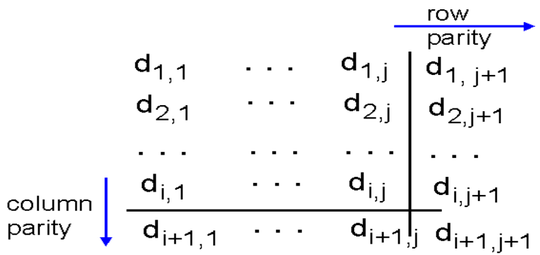
单比特奇偶校验：检测

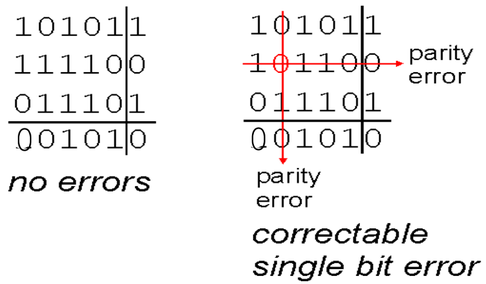
单比特错误



二维位奇偶校验：

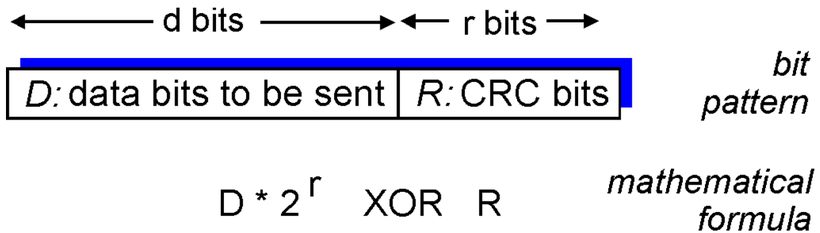
检测并纠正**单比特错误**





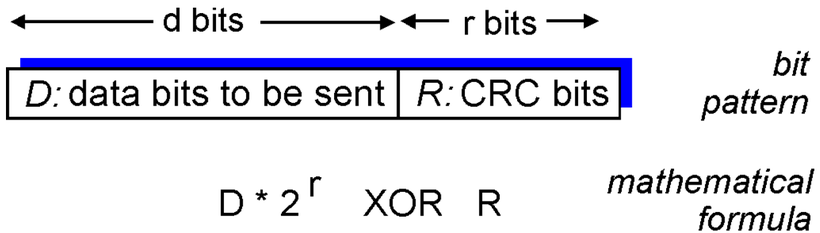
Checksumming: Cyclic Redundancy Check

- view data bits, D, as a binary number
- choose r+1 bit pattern (generator), G
- goal: choose r CRC bits, R, such that
  - <D,R> exactly divisible by G (modulo 2)
  - receiver knows G, divides <D,R> by G. If non-zero remainder: error detected!
  - can detect all burst errors less than r+1 bits
- widely used in practice (ATM, HDCL)



校验和：循环冗余校验

- 查看数据位，D，作为二进制数 选择 r+1 位模式（生成器），G 目标：选择r CRC位，R，使得
  - <D,R> 能被G（模2）整除 接收者知道G，用G除以 <D,R> 。如果余数不为零：检测到错误！ 可以检测所有长度小于r+1 位的突发错误
- 在实践中被广泛使用（ATM、HDCL）



CRC Example

Want:

D·2r XOR R = nG

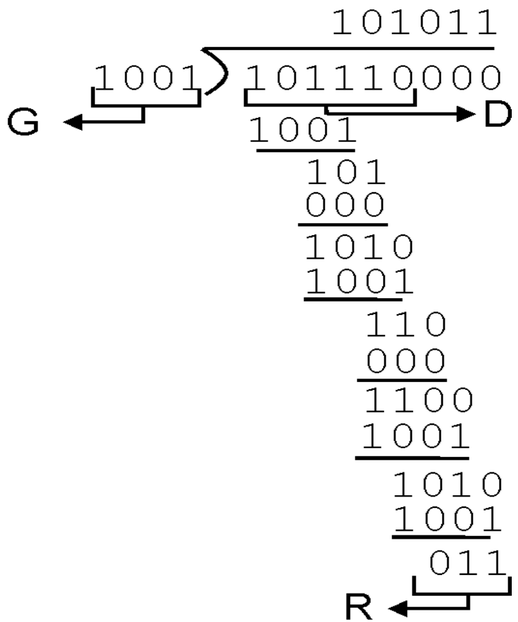
equivalently:

D·2r = nG XOR R

equivalently:

if we divide D·2r by G, want reminder R

$$R = \text{remainder}\left[\frac{D \cdot 2r}{G}\right]$$



CRC示例

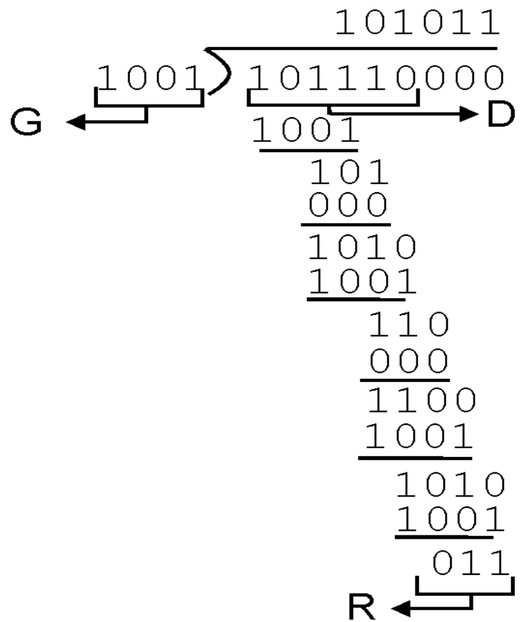
想要：

D·2r XOR R = nG

等价于：D·2r = nG

XOR R等价于：如果我们除以D·2r除以G，希望余数为R

$$R = \text{余数}\left[\frac{D \cdot 2}{r\ G}\right]$$



CRC well-known generator

- CRC-12: $x^{12}+x^{11}+x^3+x^2+x^1+1$
- CRC-16 : $x^{16}+x^{15}+x^2+1$
- CRC-CCITT : $x^{16}+x^{12}+x^5+1$
- CRC-32: $x^{32}+x^{26}+x^{23}+x^{22}+x^{16}+x^{12}+x^{11}+x^{10}+x^8+x^7+x^5+x^4+x^2+x^1+1$

CCITT: Consultative Committee on International Telegraphy and Telephone

ITU-T: International Telecommunications Union – Telecommunications Standardization Sector

CRC常用生成器

- CRC-12:$x^{12}+x^{11}+x^3+x^2+x^1+1$
- CRC-16 :$x^{16}+x^{15}+x^2+1$
- CRC-CCITT :$x^{16}+x^{12}+x^5+1$
- CRC-32: $x^{32}+x^{26}+x^{23}+x^{22}+x^{16}+x^{12}+x^{11}+x^{10}+x^8+x^7+x^5+x^4+x^2+x^1+1$

CCITT：国际电报电话咨询委员会

国际电信联盟电信标准化部门：国际电信联盟 - 电信标准化部门

3-23

Exercise-2

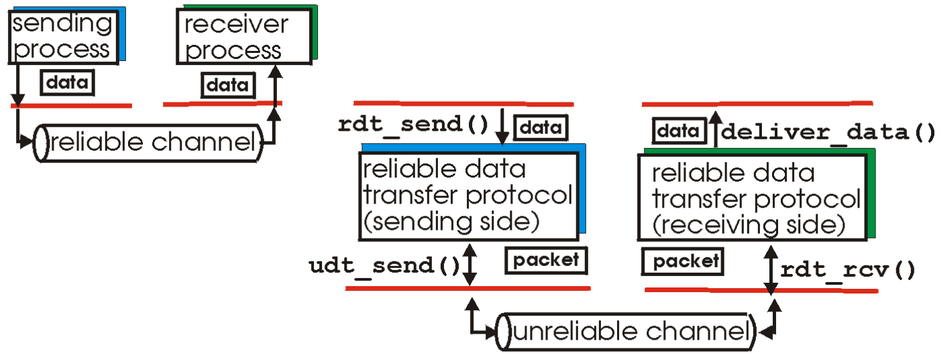
1. If the information received by an Ethernet adapter is 101101001 and the generator is $G(x) = x^3+x^2+1$, judge whether the transmission has bit error?

练习-2

1. 如果以太网适配器接收到的信息是101101001，生成多项式是 $G(x) = x^3+x^2+1$，判断传输是否有位错误?

Reliable Data Transfer

- Important in app., transport, link layers
- Top-10 list of important networking topics!

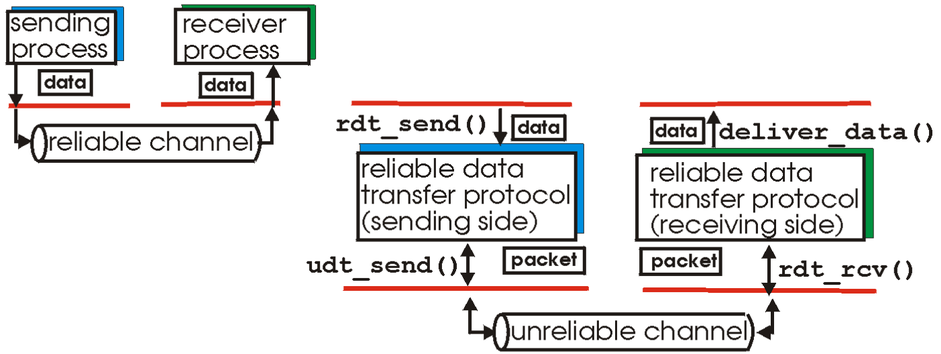


(a) provided service (b) service implementation

- Characteristics of unreliable channel will determine complexity of reliable data transfer protocol (rdt)

可靠数据传输

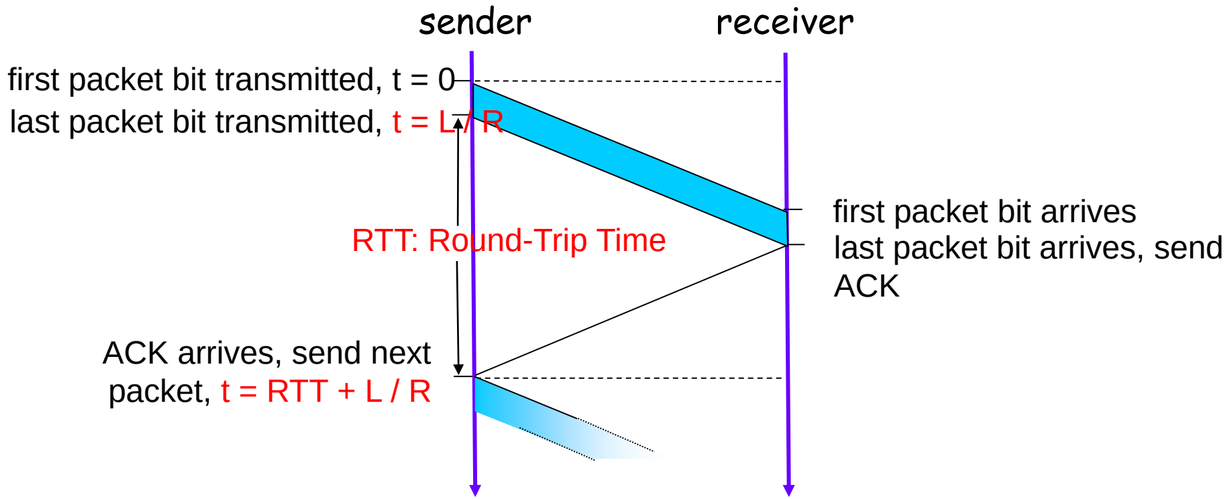
- 在应用层、传输层和链路层中都很重要，
- 重要网络主题的十大列表！



(a) provided service (b) service implementation

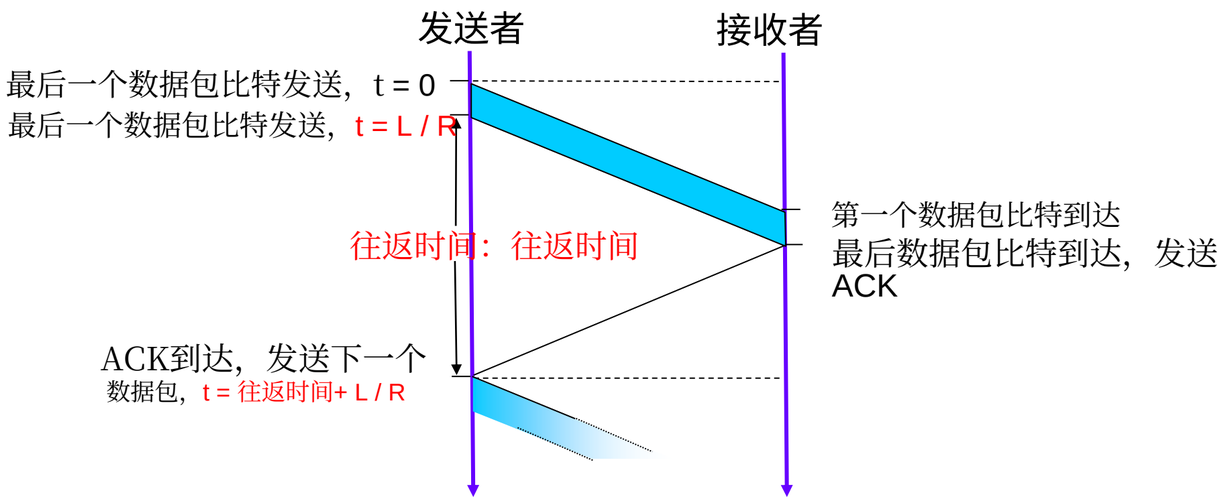
- 不可靠通道的特性将决定可靠数据传输协议（rdt）的复杂性

Stop-and-Wait



$$U_{sender} = \frac{L / R}{RTT + L / R} = \frac{.008}{30.008} = 0.00027$$

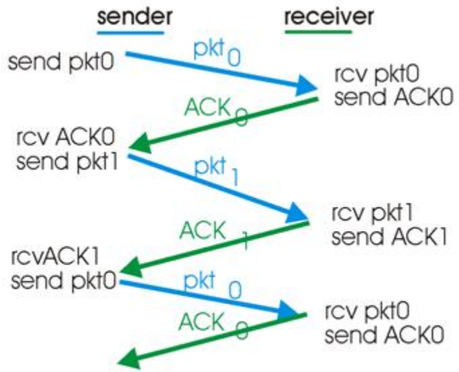
停止等待



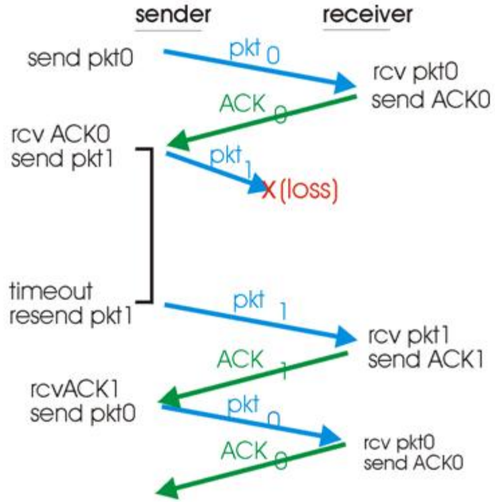
$$U_{发送者} = \frac{L/R}{往返时间 + L/R} = \frac{.008}{30.008} = 0.00027$$

微秒

Stop-and-Wait

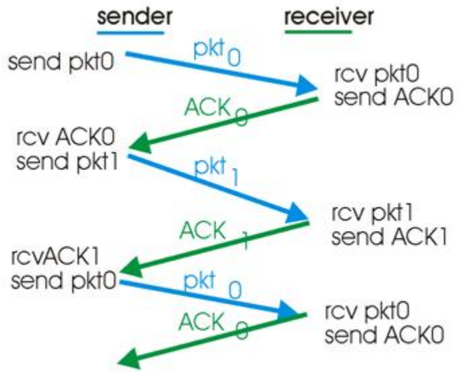


(a) operation with no loss

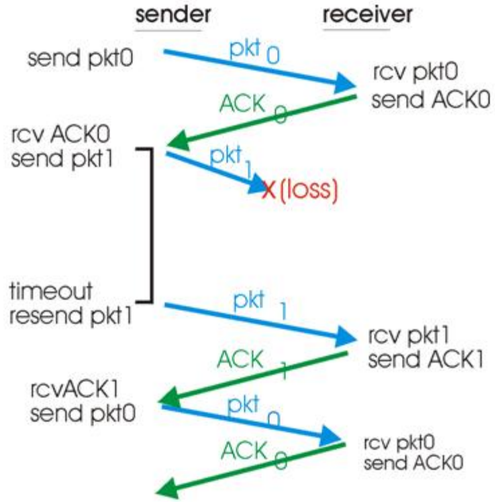


(b) lost packet

停止等待

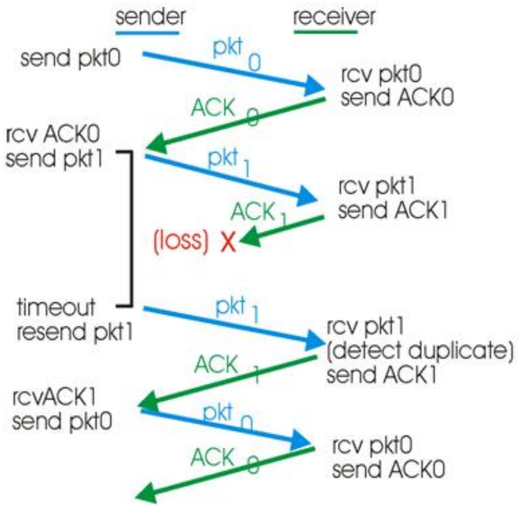


(a) operation with no loss

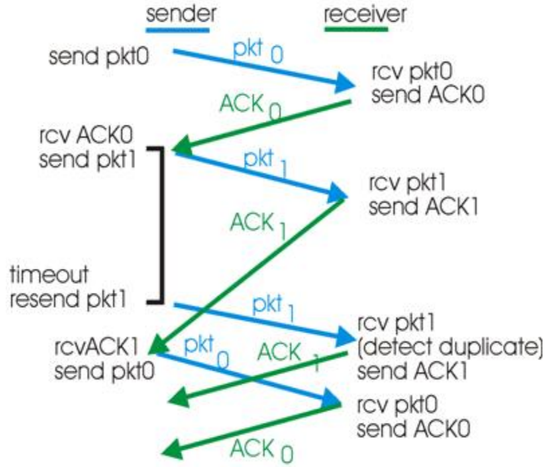


(b) lost packet

Stop-and-Wait

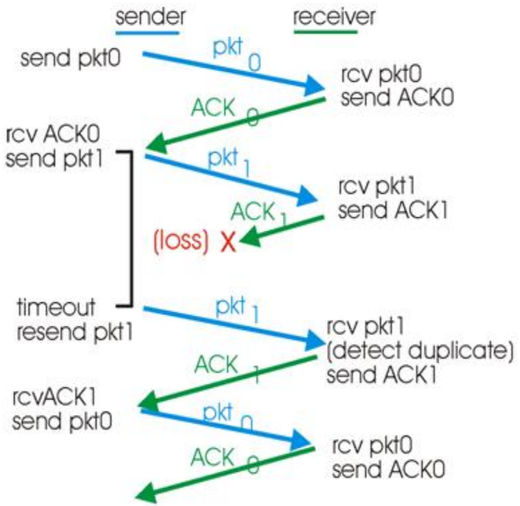


(c) lost ACK

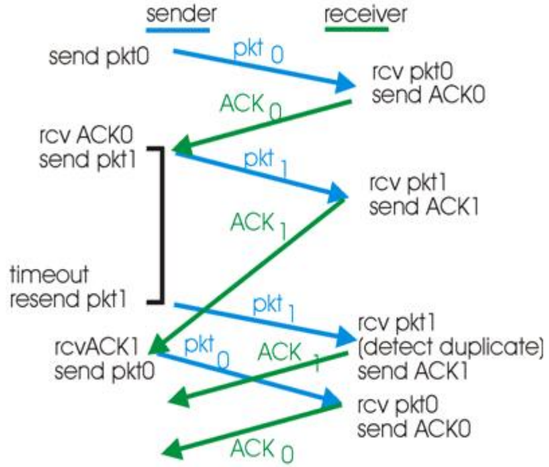


(d) premature timeout

停止等待



(c) lost ACK

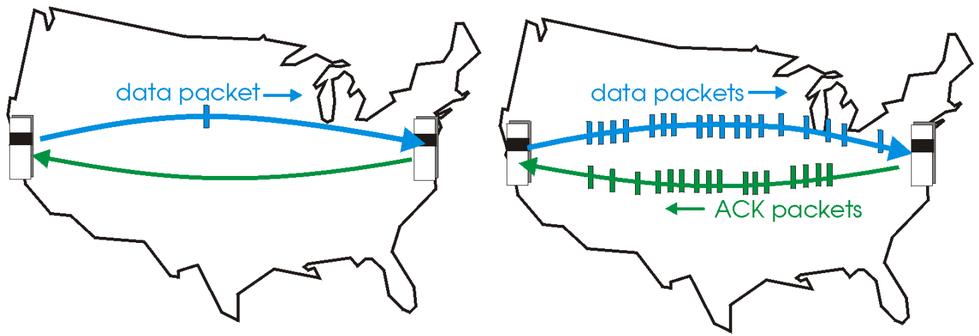


(d) premature timeout

Pipelined Protocols

Pipelining: sender allows multiple, "in-flight", yet-to-be-acknowledged pkts

- Range of sequence numbers must be increased
- Buffering at sender and/or receiver

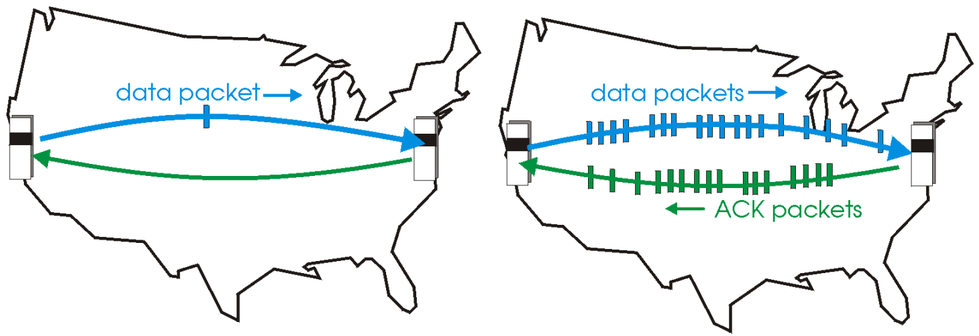


(a) a stop-and-wait protocol in operation (b) a pipelined protocol in operation

流水线协议

流水线：发送者允许多个，“-在途”，尚未确认的数据包

- 序列号的范围必须增加
- 发送者和/或接收者处的缓冲



(a) a stop-and-wait protocol in operation (b) a pipelined protocol in operation

Pipelined Protocols

- Advantage: much better bandwidth utilization than stop-and-wait
- Disadvantage: More complicated to deal with reliability issues, e.g., corrupted, lost, out of order data.
  - Two generic approaches to solving this
    - *Go-Back-N protocols*
    - *Selective repeat protocols*
- Note: *TCP is not exactly either*

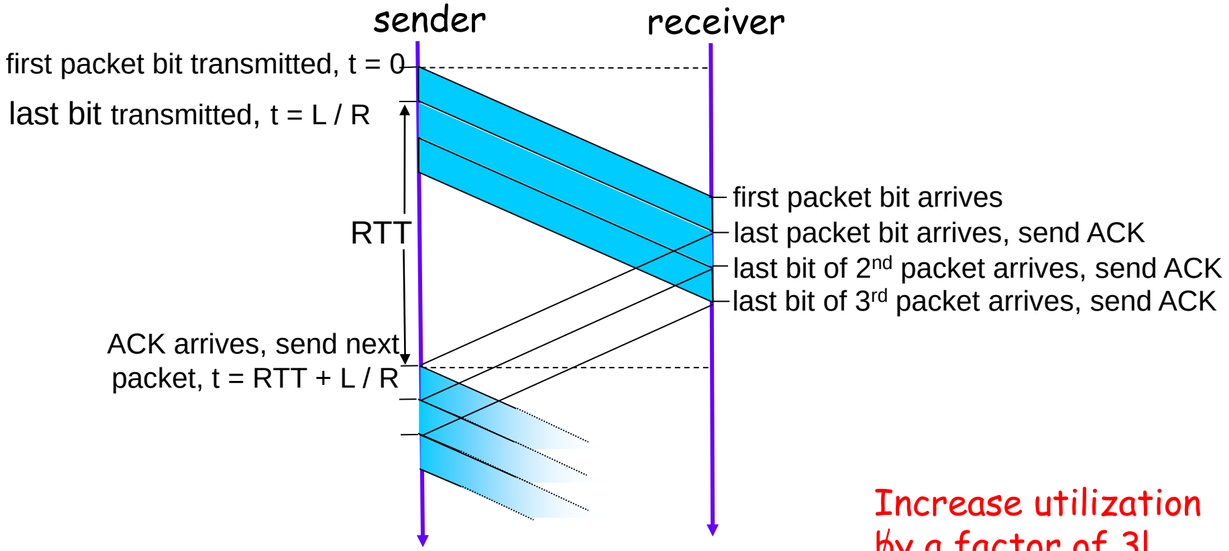


流水线协议

- 优点：比停止等待机制有更好的带宽利用率
- 缺点：处理可靠性问题更复杂，例如损坏、丢失、乱序数据。
  - 解决此问题的两种通用方法
    - 回退N 协议
    - 选择重传 协议
- 注意： TCP并不完全是其中任何一种



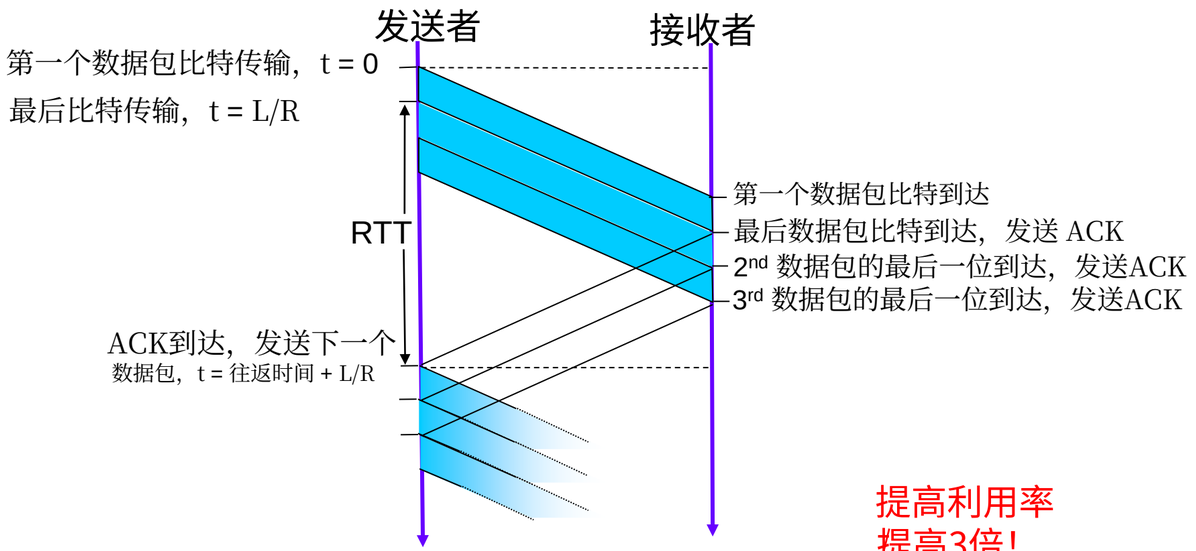
Pipelining: Increased Utilization



Increase utilization by a factor of 3!

$$U_{sender} = \frac{3 * L / R}{RTT + L / R} = \frac{.024}{30.008} = 0.0008$$

流水线：提高利用率



提高利用率提高3倍！

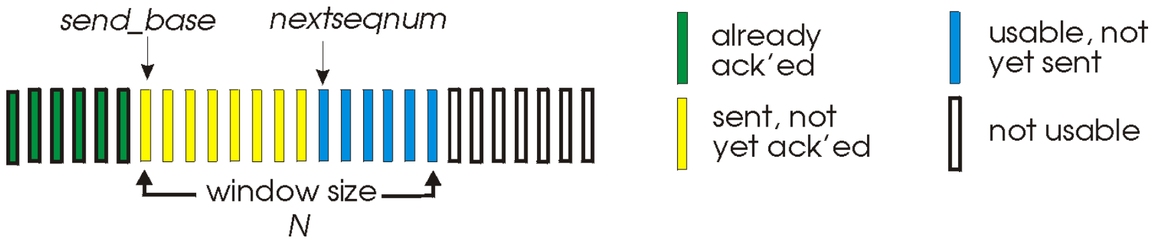
$$U_{发送者} = \frac{3 * L/R}{往返时间 + L/R} = \frac{.024}{30.008} = 0.0008$$

微秒

Go-Back-N

Sender:

- K-bit seq # in pkt header
- "Window" of up to N, consecutive unack'ed pkts allowed

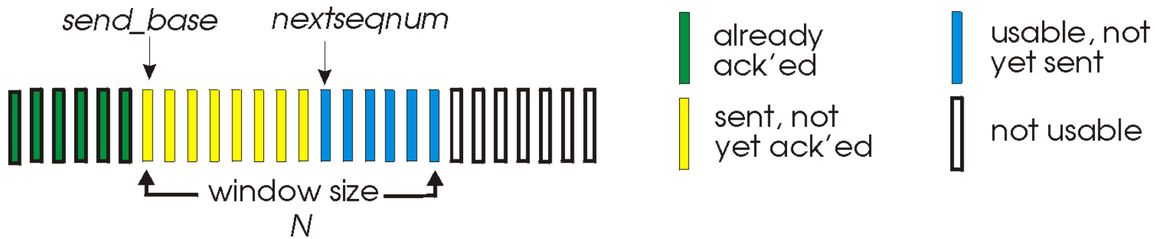


- ACK(n): ACKs all pkts up to, including seq # n - "cumulative ACK"
  - May receive duplicate ACKs (see receiver)
- Timer for each in-flight pkt
- *Timeout(n)*: retransmit pkt n and all higher seq # pkts in window
- Called a sliding-window protocol

回退N

发送者：

- 包头中的K位序列号
- "窗口"大小最多为N，允许连续未确认的包



- ACK(n)：确认序列号n及之前的所有包——"累积确认"
  - 可能收到重复的ACK（见接收者）
- 每个在飞包的计时器
- 超时(n)：重传序列号n及窗口中所有更高序列号的包
- 被称为滑动窗口协议

GBN: Sender

- rdt_Send() called: checks to see if window is full
  - No: send out packet
  - Yes: return data to upper level
- Receipt of ACK(n): cumulative acknowledgement that all packets up to and including *n* have been received. Updates window accordingly and restarts timer
- Timeout: resends ALL packets that have been sent but not yet acknowledged.
  - This is only event that triggers resend

GBN: 发送者

- rdt_发送()被调用：检查窗口是否已满
  - 否： 发送数据包 是： 返回数据至上层
- ACK(n)的接收： 累积确认，表示所有编号至包括*n* 的数据包均已接收。相应更新窗口并重启计时器
- 超时： 重发所有已发送但尚未确认的数据包
  - 这是唯一会触发重发的场景

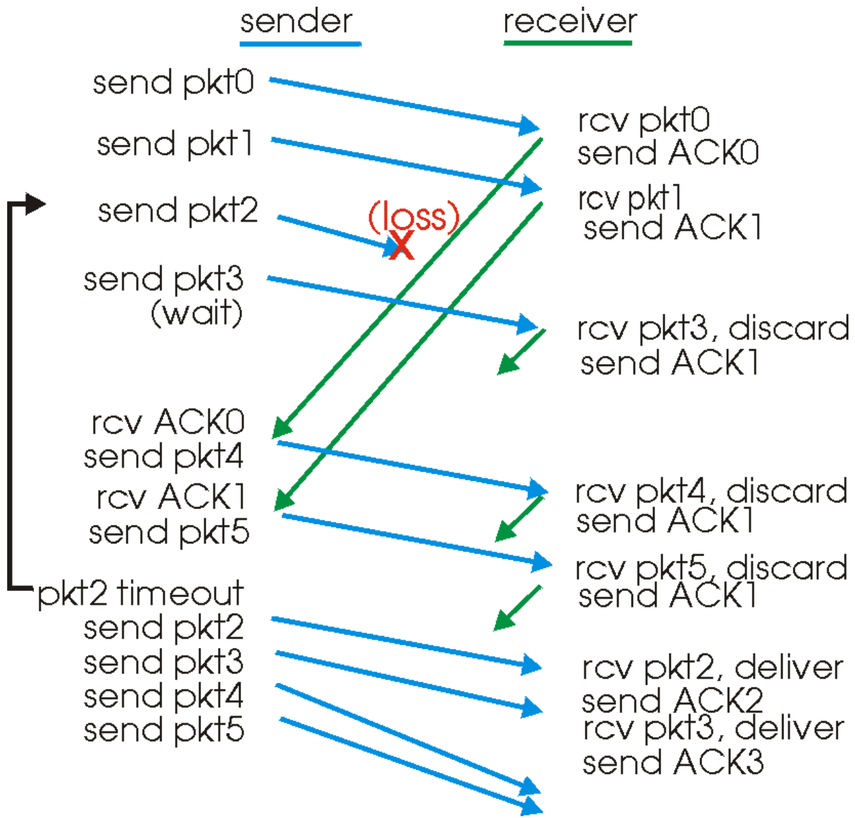
More on Receiver

- ACK-only: the receiver always sends ACK for last correctly received packet with highest *in-order* seq #
- Receiver only sends ACKS (no NAKs)
- Need only remember **`expectedseqnum`**
- Can generate duplicate ACKs

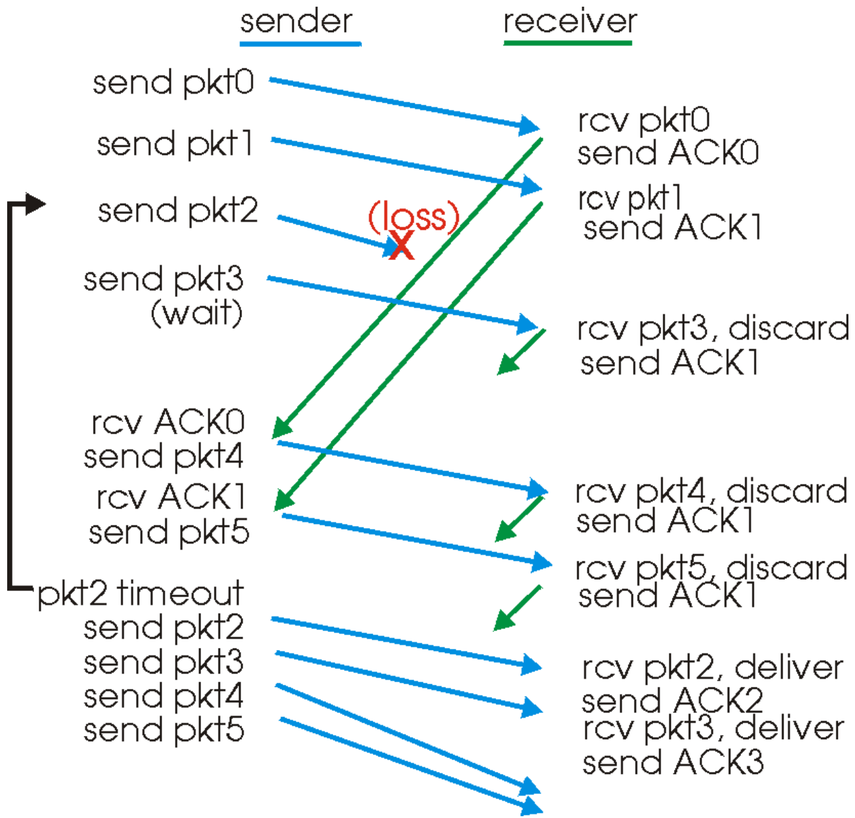
更多关于接收者

- 仅ACK：接收者总是对最后正确接收的数据包发送ACK，且使用最高有序序列号
- 接收者仅发送ACK（不发送NAK）
- 只需记住**预期序列号**
- 可能生成重复的ACK

GBN In Action



GBN 实战



- GBN is easy to code but might have performance problems

- In particular, if many packets are in pipeline at one time (bandwidth-delay product large) then one error can force retransmission of huge amounts of data!

- Selective Repeat protocol allows receiver to buffer data and only forces retransmission of required packets

- GBN 编写起来容易，但可能会出现性能问题

- 特别是，如果一次有大量数据包在管道中（带宽时延积很大）那么一个错误就可能导致大量数据的重传！

- 选择性重传协议允许接收者缓存数据，并且仅强制重传所需的数据包

3-36

Selective Repeat

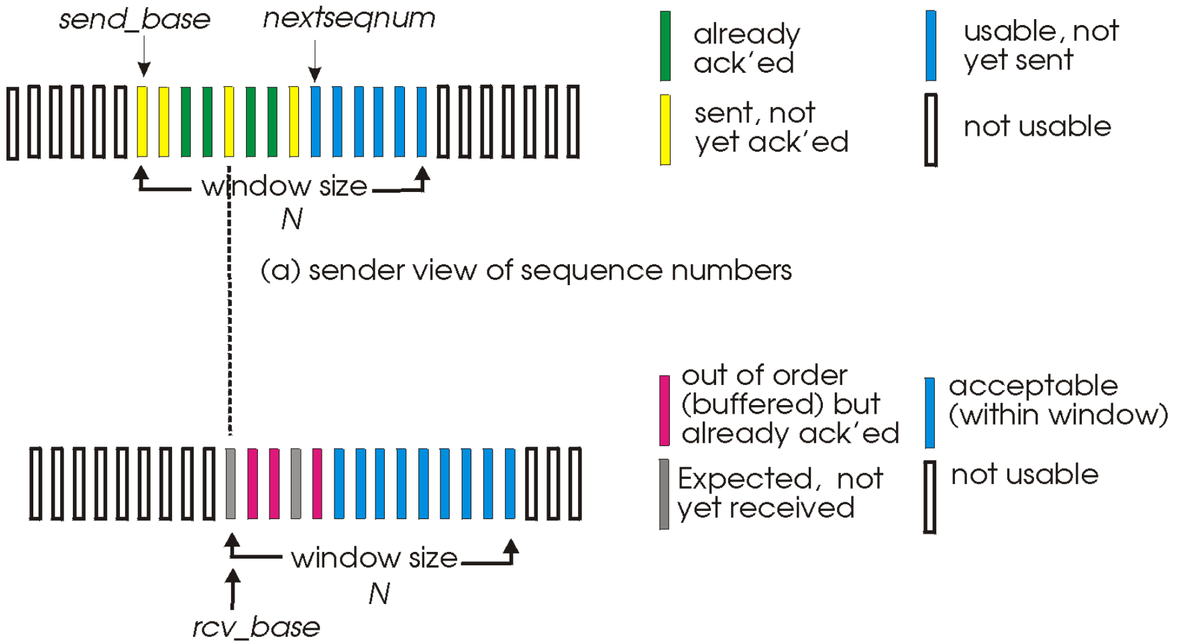
- Receiver *individually* acknowledges all correctly received pkts
  - Buffers pkts, as needed, for eventual in-order delivery to upper layer
- Sender only resends pkts for which ACK not received
  - Sender timer for each unACKed pkt
  - Compare to GBN which only had timer for base packet
- Sender window
  - N consecutive seq #'s
  - Again limits seq #s of sent, unACKed pkts
  - Important: Window size < seq # range

选择性重传

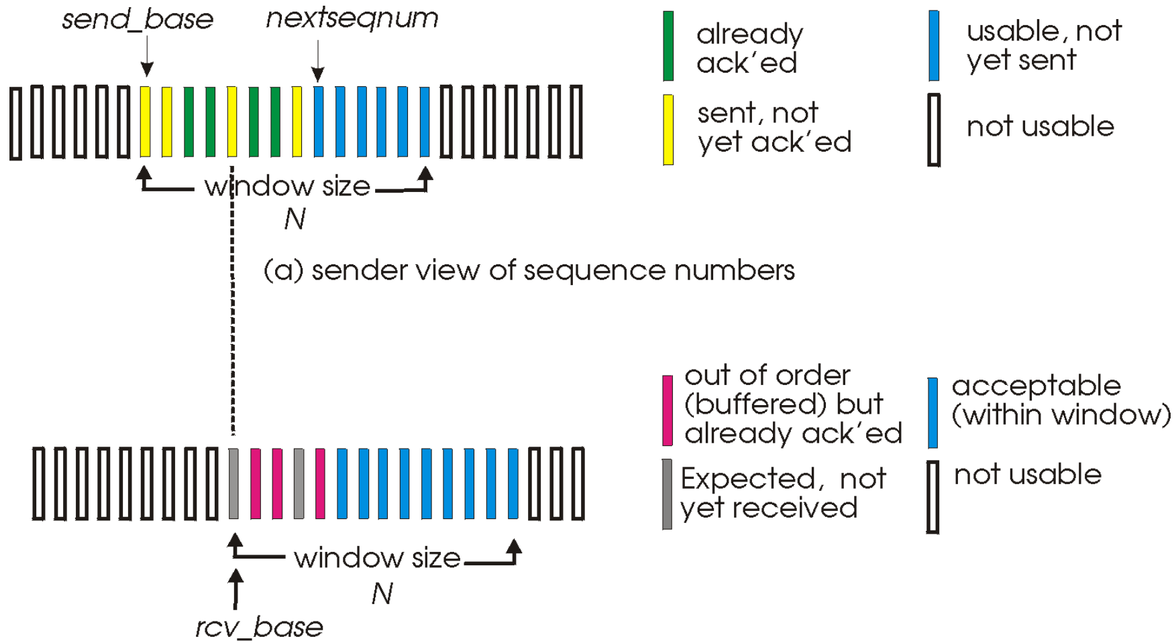
- 接收者 单独确认所有正确接收的数据包
  - 根据需要缓存数据包，以便最终按顺序向上层交付
- 发送者仅重传未收到ACK的数据包
  - 发送者计时器用于每个未ACK的数据包 与GBN相比，GBN仅对基础数据包使用计时器
- 发送者窗口
  - N个连续的序列号 再次限制已发送、未ACK的数据包的序列号 重要：窗口大小< 序列号范围

3-37

Selective Repeat: Sender, Receiver Windows



S可选 重复：发送者、接收者窗口



Selective Repeat

sender

Data from above :

- If next available seq # in window, send pkt

Timeout(n):

- Resend pkt n, restart timer

ACK(n) in [sendbase,sendbase+N]:

- Mark pkt n as received
- If n smallest unACKed pkt, advance window base to next unACKed seq #

receiver

pkt n in [rcvbase, rcvbase+N-1]

- Send ACK(n)
- Out-of-order: buffer
- In-order: deliver (also deliver buffered, in-order pkts), advance window to next not-yet-received pkt

pkt n in [rcvbase-N,rcvbase-1]

- ACK(n)

Otherwise:

- Ignore

选择性重传

发送者

来自上方的数据：

- 如果窗口中的下一个可用序列号，发送数据包

超时(n)：

- 重新发送数据包n，重启计时器

ACK(n) 在 [发送基准，发送基准+N]:

- 将数据包n标记为已接收
- 如果n是最小的未ACK数据包，则将窗口基址移动到下一个未ACK序列号

接收者

数据包n在 [接收基准，接收基准+N-1]

- 发送ACK(n)
- 乱序：缓冲
- 有序：传递（同时传递缓冲的有序数据包），将窗口推进到下一个未接收的数据包

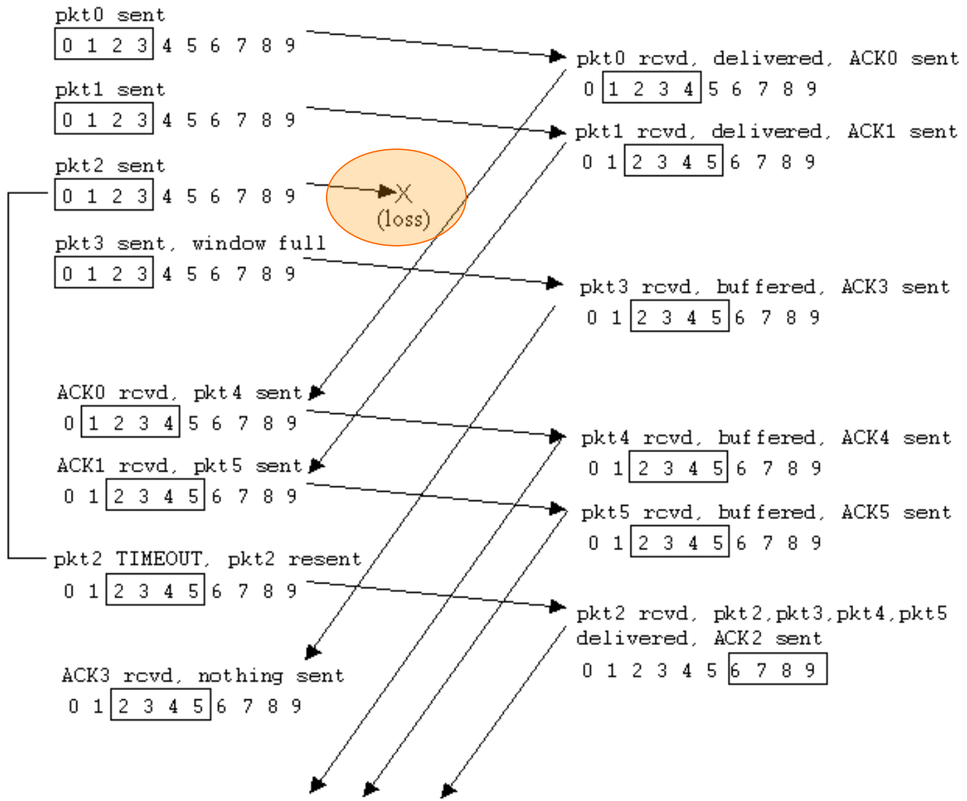
数据包n在 [接收基准-N，接收基准-1]

- ACK(n)

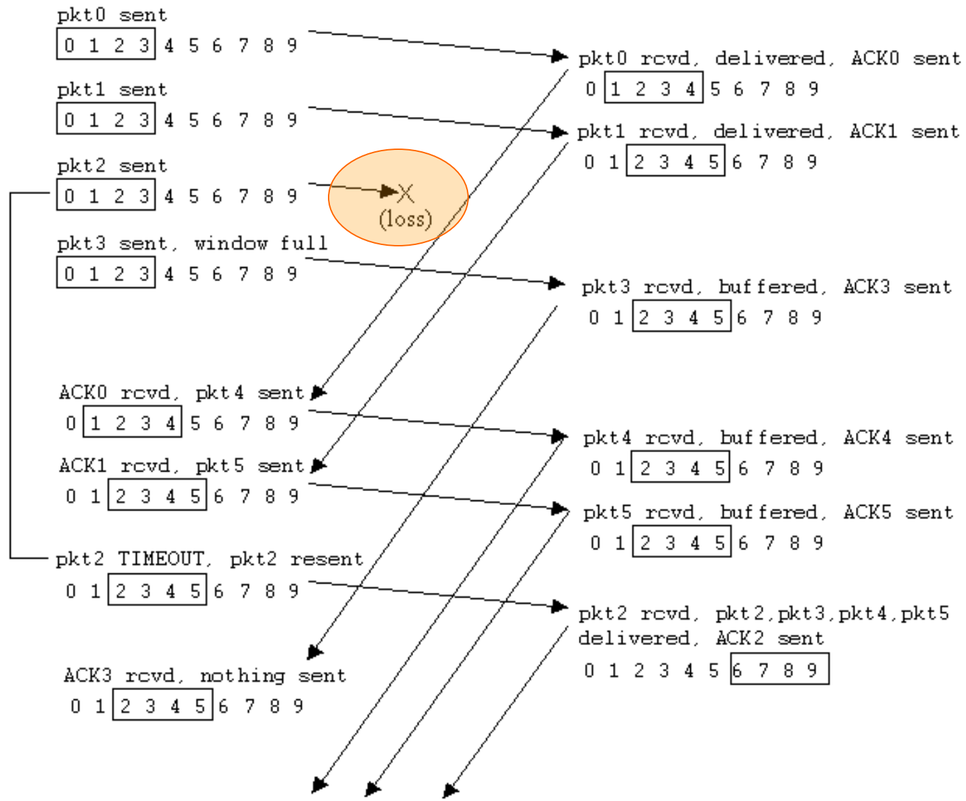
否则：

- 忽略

Selective Repeat In Action



选择性重传 运行中

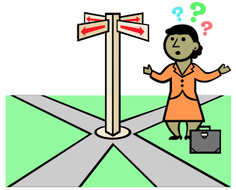


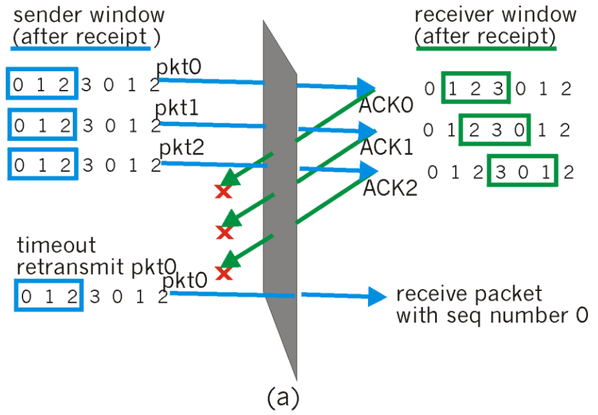
Selective Repeat: Dilemma

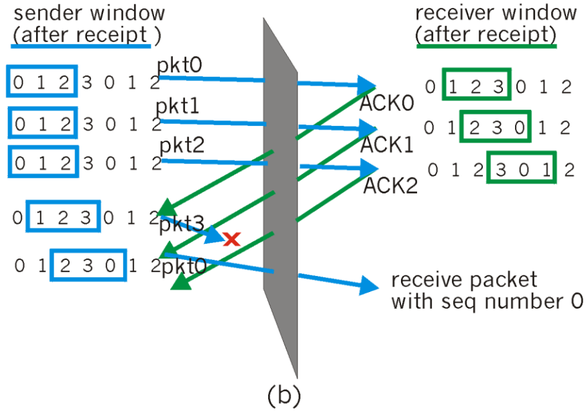
Example:

- seq #'s: 0, 1, 2, 3
- window size=3
- Receiver sees no difference in two scenarios!
- Incorrectly passes duplicate data as new in (a)

Q: what relationship between seq # size and window size?





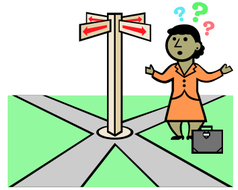


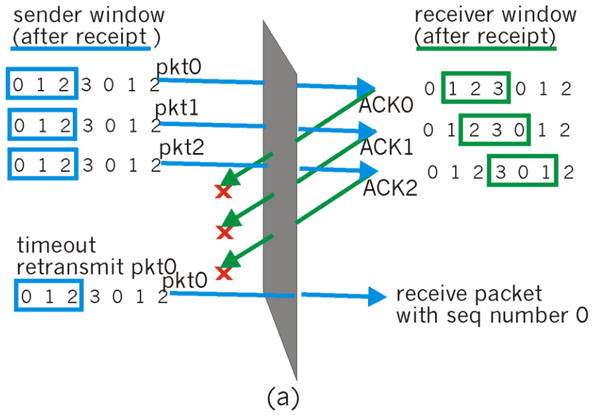
选择性重传：困境

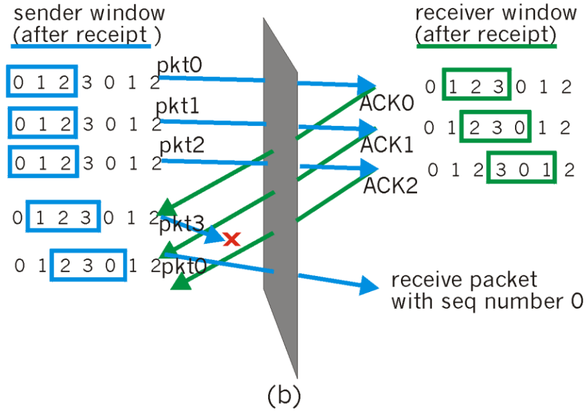
示例：

- 序列号：0，1，2，3
- 窗口大小=3
- 接收者在两种场景下没有区别！
- (a)中错误地将重复数据当作新数据传递

Q: 序列号大小和窗口大小之间有什么关系?

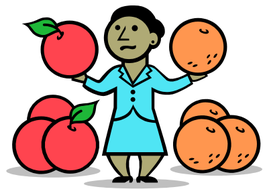






GBN vs. Selective Repeat

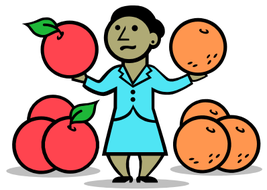
- Selective repeat is more complicated as it needs buffering at the receiver, but only retransmit packets required for retransmission



- GBN is simpler, but can lead to large number of unnecessary retransmission

GBN 与 选择性重传

- 选择性重传更复杂，因为它需要在接收者处进行缓冲，但仅重传需要重传的数据包



- GBN 更简单，但可能导致大量不必要的重传

Exercises-3

1. The data link layer uses the GBN protocol, and the sender has sent frames numbered 0 ~ 7. When the timer expires, if the sender only receives the acknowledgement of frames 0, 2 and 3, the number of frames to be retransmitted by the sender is ( )

A. 2 B. 3 C. 4 D.5

练习-3

1. 数据链路层使用GBN协议，发送者已发送编号为0 ~ 7的帧。当计时器超时时，如果发送者仅收到帧0、2和3的确认，发送者需要重传的帧数量是 ( )

A. 2 B. 3 C. 4 D.5

Exercise-3

2. Considering GBN protocol and SR protocol, assuming that the length of sequence number space is *n*, what is the maximum allowed sender window?

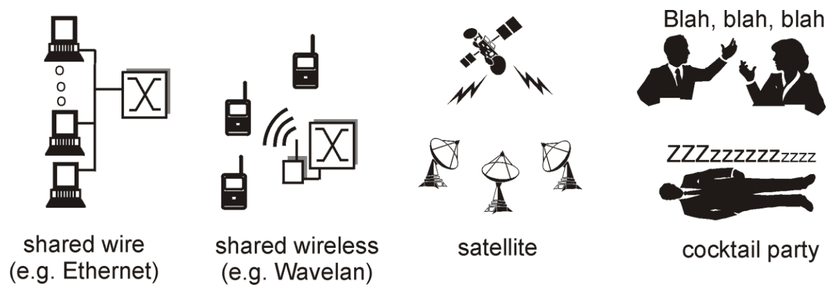
练习-3

2. 考虑GBN协议和SR协议，假设序列号空间的长度是*n*，发送者的最大允许窗口是多少?

Multiple Access Links and Protocols

Three types of "links":

- point-to-point (single wire, e.g. PPP, SLIP)
- broadcast (shared wire or medium; e.g, Ethernet, Wavelan, etc.)

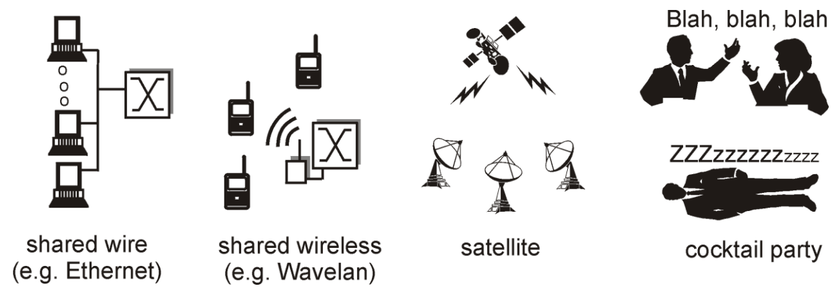


- switched (e.g., switched Ethernet, ATM etc)

多点接入链路和协议

三种“链路”类型：

- 点对点（单线，例如PPP、SLIP）
- 广播(共享线或介质；例如以太网、Wavelan等)



- 交换式（例如，交换式以太网、ATM 等）

Multiple Access protocols

- single shared communication channel
- two or more simultaneous transmissions by nodes: interference
  - collision if node receives two or more signals at the same time
  - only one node can send successfully at a time
- *multiple access protocol:*
  - distributed algorithm that determines how nodes share channel, i.e., determine when node can transmit
  - communication about channel sharing must use channel itself!
  - what to look for in multiple access protocols:
    - synchronous or asynchronous
    - information needed about other stations
    - robustness (e.g., to channel errors)
    - performance

多路访问协议

- 单一共享通信通道
- 两个或多个节点同时发送：干扰
  - 如果节点在同一时间接收到两个或多个信号，则发生冲突
  - 一次只能有一个节点发送成功地
- 多路访问协议：
  - 一种分布式算法，用于确定节点如何共享通道，即确定节点何时可以传输
  - 关于通道共享的通信必须使用通道本身 !
  - 多路访问协议中需要关注的内容：
    - 同步或异步
    - 关于其他站点的信息
    - 鲁棒性（例如，对通道错误的鲁棒性）
    - 性能

Multiple Access protocols

- claim: humans use multiple access protocols all the time
- class can "guess" multiple access protocols
  - multiaccess protocol 1:
  - multiaccess protocol 2:
  - multiaccess protocol 3:
  - multiaccess protocol 4:

多路访问协议

- 声明：人类一直使用多路访问协议
- 类可以"猜测"多路访问协议
  - 多路访问协议 1:
  - 多路访问协议 2:
  - 多路访问协议 3:
  - 多路访问协议 4:

MAC Protocols: a taxonomy

Three broad classes:

- Channel Partitioning
  - divide channel into smaller "pieces" (time slots, frequency)
  - allocate piece to node for exclusive use
- Random Access
  - allow collisions
  - "recover" from collisions
- "Taking turns"
  - tightly coordinate shared access to avoid collisions

Goal: efficient, fair, simple, decentralized

MAC 协议：分类学

三大类：

- 信道划分
  - 将信道划分为更小的“片段”（时隙、频段）
  - 为节点分配片段以独占使用
- 随机接入
  - 允许冲突
  - “从冲突中恢复”
- **“轮流”**
  - 紧密协调共享访问以避免冲突

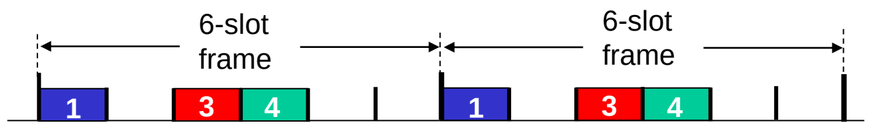
目标：高效、公平、简单、去中心化

3-48

Channel Partitioning MAC protocols: TDMA

TDMA: time division multiple access

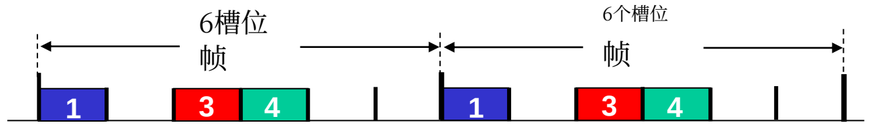
- access to channel in "rounds"
- each station gets fixed length slot (length = pkt trans time) in each round
- unused slots go idle
- example: 6-station LAN, 1,3,4 have pkt, slots 2,5,6 idle



信道划分MAC协议：TDMA

TDMA：时分多址

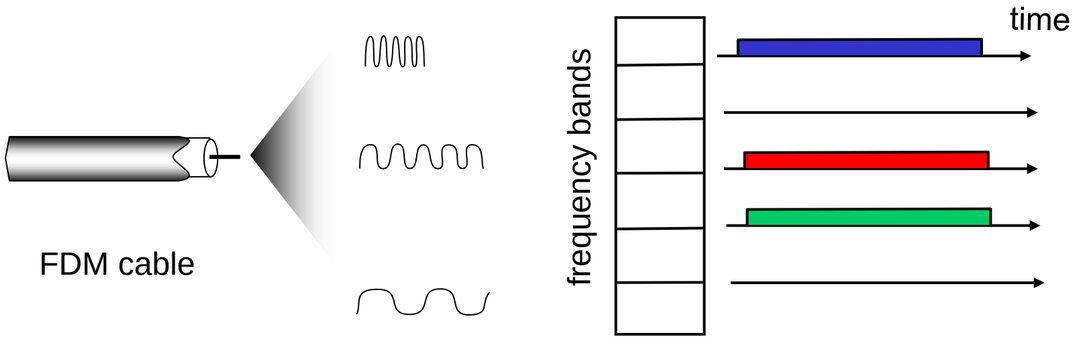
- 在"轮次"中访问通道
- 每个站点在每个轮次中获得固定长度的槽位（长度= 数据包传输时间）
- 未使用的槽位处于空闲状态
- 示例：6站LAN，1、3、4有数据包，2、5、6槽位空闲



Channel Partitioning MAC protocols: FDMA

FDMA: frequency division multiple access

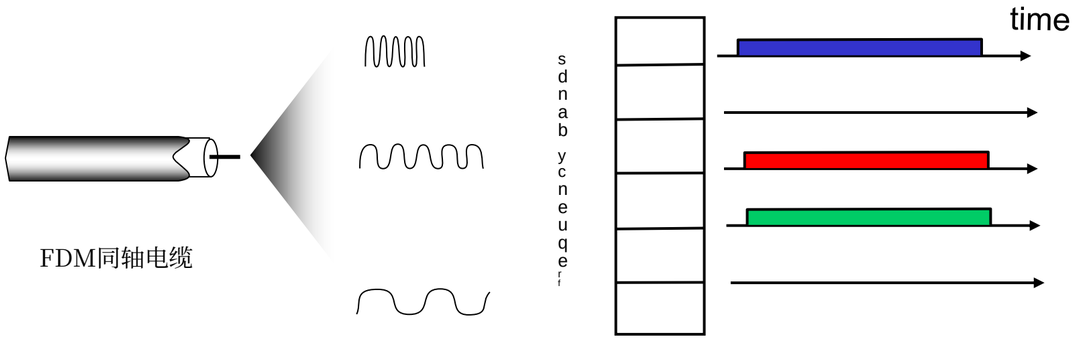
- channel spectrum divided into frequency bands
- each station assigned fixed frequency band
- unused transmission time in frequency bands go idle
- example: 6-station LAN, 1,3,4 have pkt, frequency bands 2,5,6 idle



信道划分MAC协议：FDMA

FDMA：频分多址

- 通道频谱划分为频段
- 每站分配固定频段
- 频段中未使用的传输时间空闲
- 示例：6站LAN，1、3、4有数据包，频段2、5、6空闲



Code Division Multiple Access (CDMA)

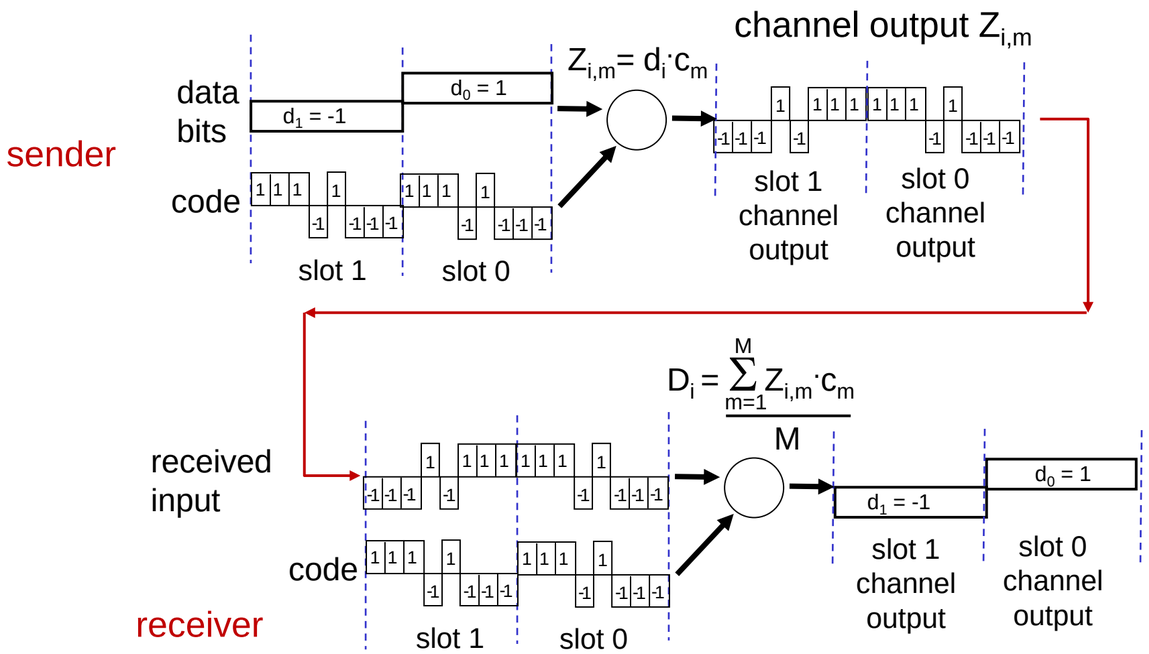
- unique "code" assigned to each user; i.e., code set partitioning
  - all users share same frequency, but each user has own "chipping" sequence (i.e., code) to encode data
  - allows multiple users to "coexist" and transmit simultaneously with minimal interference (if codes are "orthogonal")
- *encoded signal* = (original data) X (chipping sequence)
- *decoding:* inner-product of encoded signal and chipping sequence

码分多址 (CDMA)

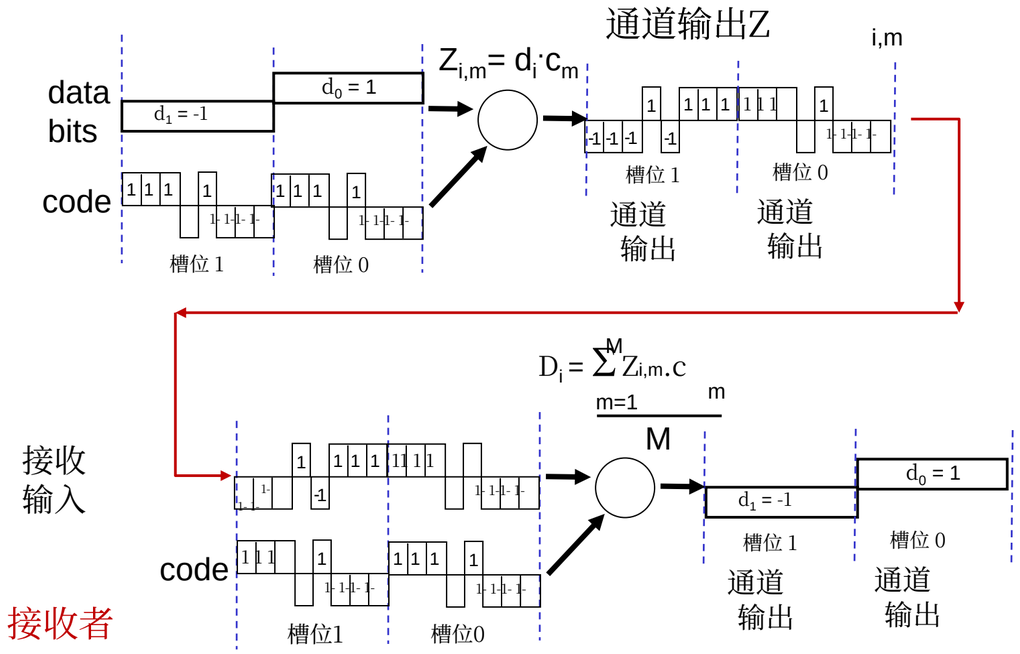
- 为每个用户分配唯一的"代码"；即，代码集划分
  - 所有用户共享相同频率，但每个用户都有自己的"码片"序列（即，代码）来编码数据
  - 允许多个用户"共存"，并以最小干扰同时传输（如果代码是"正交"的）
- 编码信号 = (原始数据) X (码片序列)
- 解码： 编码信号与码片序列的内积

3-51

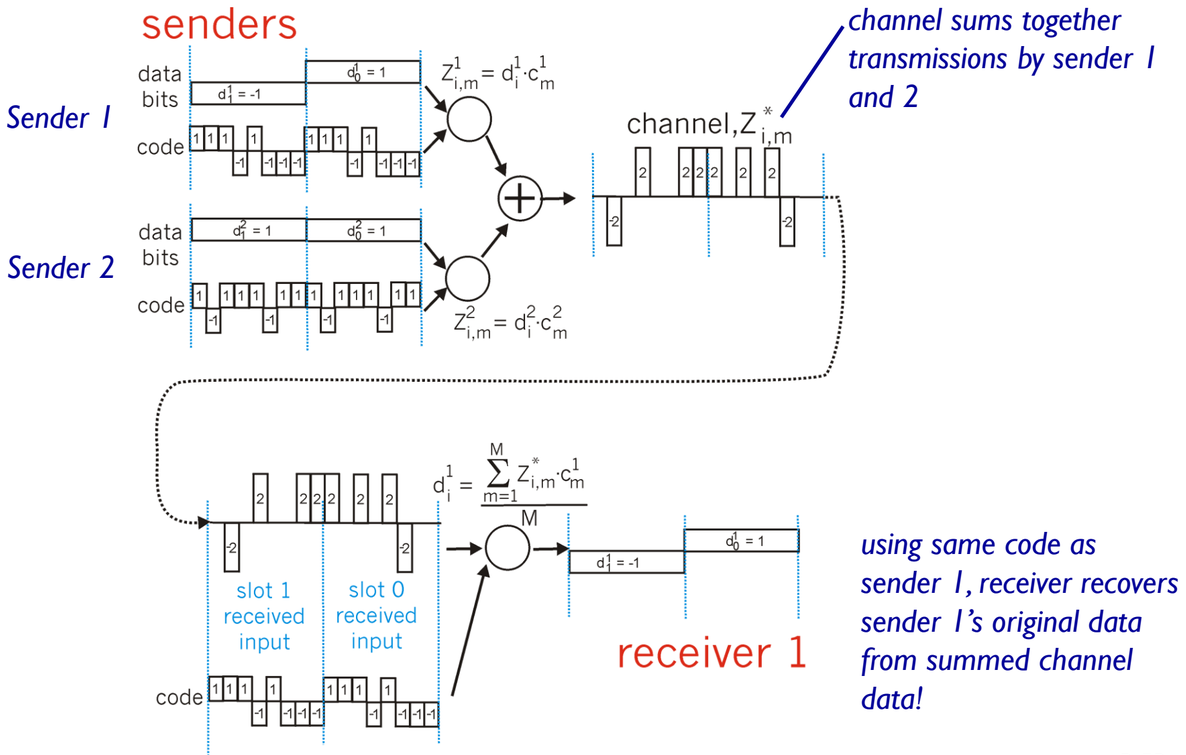
CDMA encode/decode



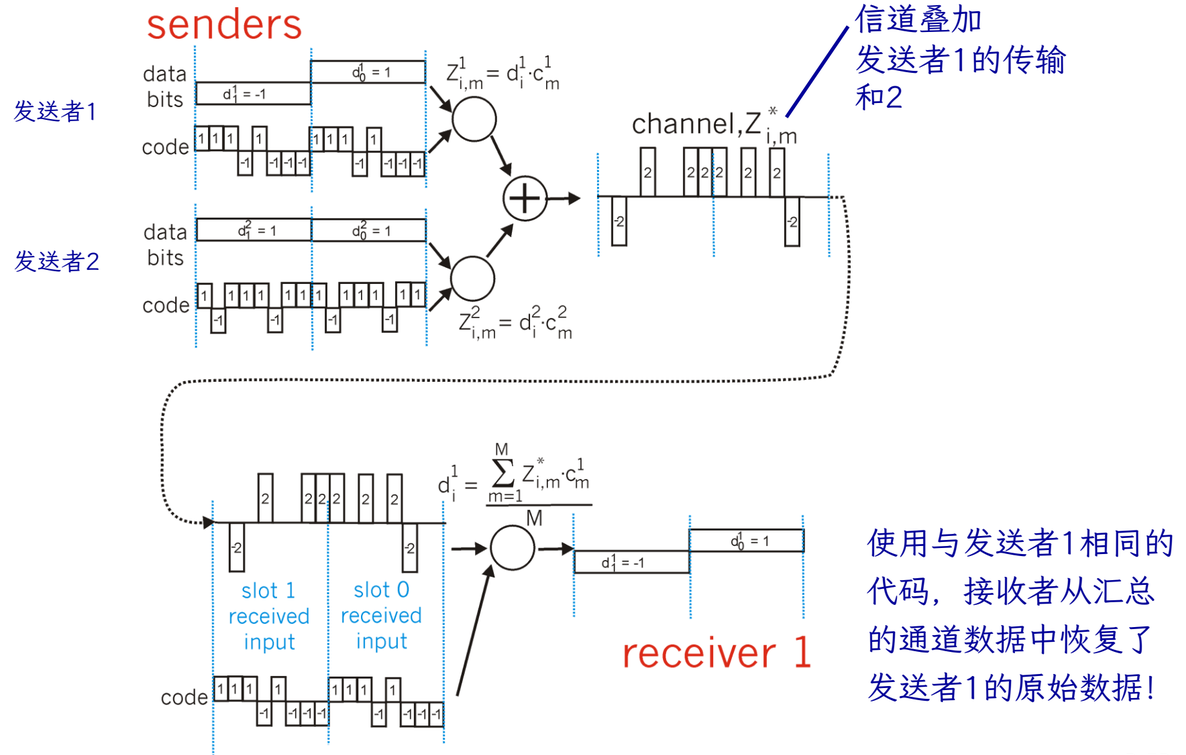
码分多址编码/解码



CDMA: two-sender interference



码分多址：两个-发送者干扰



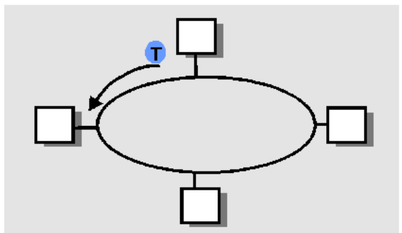
"Taking Turns" MAC protocols

Polling(轮询):

- master node "invites" slave nodes to transmit in turn
- Request to Send, Clear to Send msgs
- concerns:
  - polling overhead
  - latency
  - single point of failure (master)

Token passing:

- control **token** passed from one node to next sequentially.
- token message
- concerns:
  - token overhead
  - latency
  - single point of failure (token)



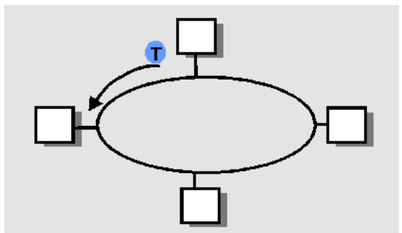
"轮流MAC协议

轮询(轮询):

- 主节点"邀请"从节点轮流传输
- 请求发送，清除发送消息
- 关注点:
  - 轮询开销 延迟
  - 单点故障（主节点）

令牌传递:

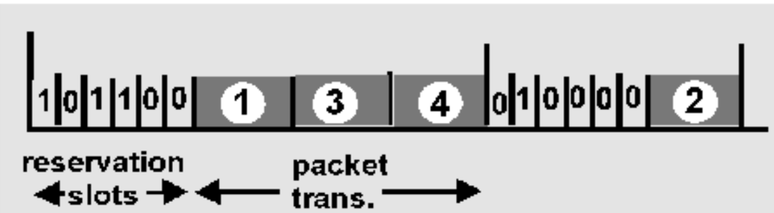
- 控制 **令牌** 按顺序从一个节点传递到下一个节点。
- 令牌消息
- 关注:
  - 令牌开销 延迟 单点故障（令牌）



Reservation-based protocols

Distributed Polling(分布式轮询):

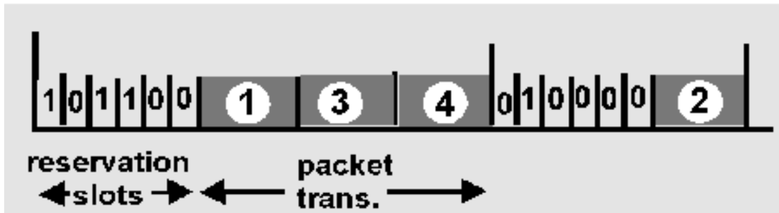
- time divided into slots
- begins with N short reservation slots
  - reservation slot time equal to channel end-end propagation delay
  - station with message to send posts reservation
  - reservation seen by all stations
- after reservation slots, message transmissions ordered by known priority



预约-基于协议

分布式轮询(分布式轮询):

- 时间划分为槽位 以 N 个短预留槽位
  - 预约槽位时间等于通道端到端传播延迟 有消息要发送的站点发布预约 所有站点都能看到预约
- 预约槽位后，消息传输按已知优先级排序



Random Access protocols

- When node has packet to send
  - transmit at full channel data rate R.
  - no *a priori* coordination among nodes
- two or more trasnmitting nodes -> "collision",
- random access MAC protocol specifies:
  - how to detect collisions
  - how to recover from collisions (e.g., via delayed retransmissions)
- Examples of random access MAC protocols:
  - ALOHA
  - slotted ALOHA
  - CSMA and CSMA/CD

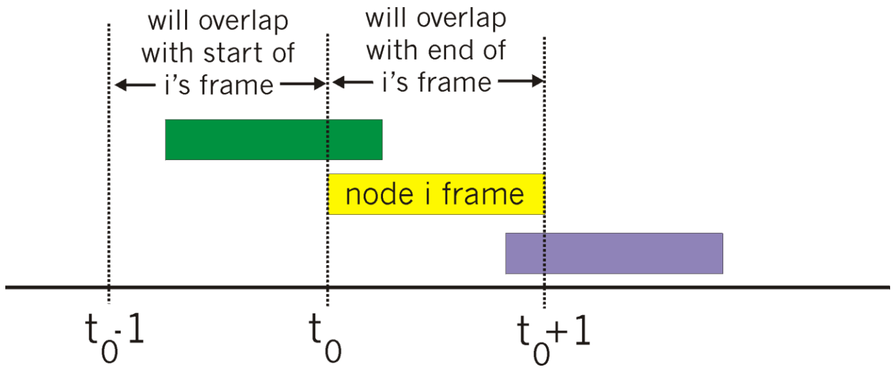
随机接入协议

- 当节点有数据包要发送时
  - 以信道数据速率 R 进行传输，节点之间没有<样式 id='3'>先验</样式>协调
- 两个或多个传输节点 -> “冲突”，
- <样式 id='2'>随机接入MAC协议</样式><样式 id='4'>规定了：</样式>
  - 如何检测冲突 如何从冲突中恢复（例如，通过延迟重传）
- 随机接入MAC协议的示例：
  - ALOHA 时隙ALOHA
  - CSMA和CSMA/CD

3-56

Pure ALOHA

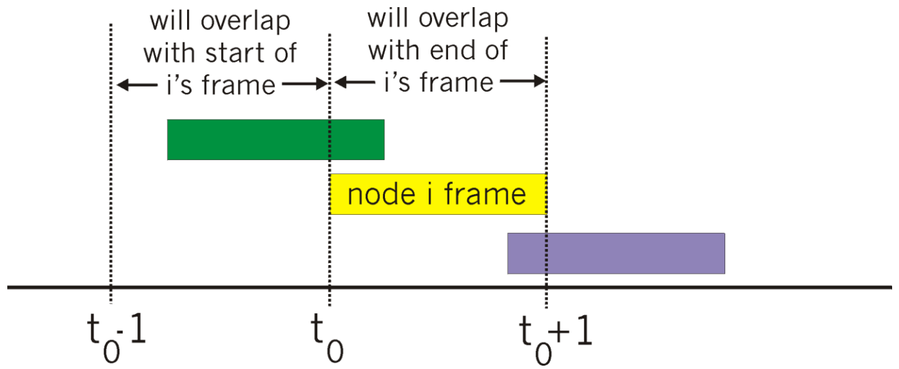
- unslotted Aloha: simpler, no synchronization
- pkt needs transmission:
  - send without awaiting for beginning of slot
- collision probability increases:
  - pkt sent at t_0 collide with other pkts sent in $[t_0-1, t_0+1]$



纯ALOHA

- 非时隙ALOHA：更简单，无需同步
- 数据包需要传输：
  - 在槽位开始前发送
- 冲突概率增加：
  - 在t时刻发送的包o与在[to-1、t_0+1时刻发送的其他包发生冲突

]



Pure Aloha (cont.)

P(success by given node) = P(node transmits) ·

P(no other node transmits in $[p_0\text{-}1,p_0]$ ·

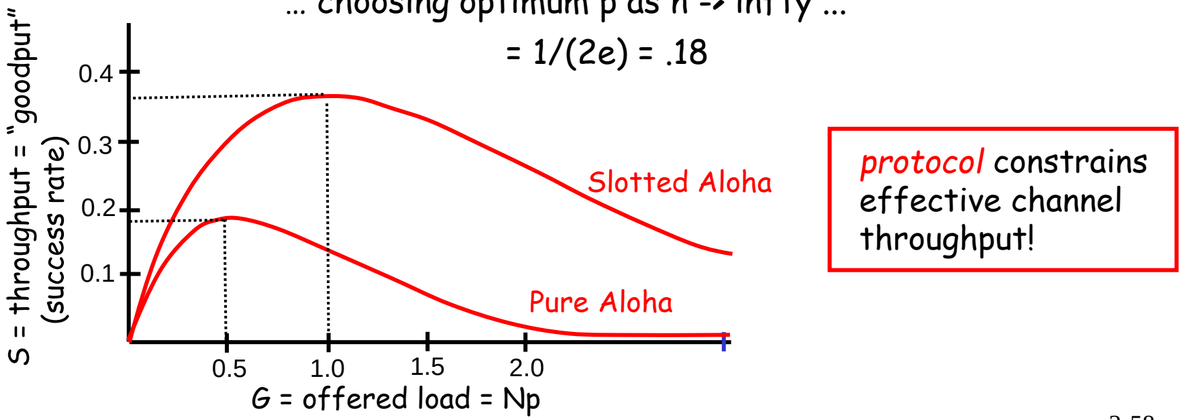
P(no other node transmits in $[p_0\text{-}1,p_0]$

= $p \cdot (1\text{-}p)^{N-1} \cdot (1\text{-}p)^{N-1}$

= $p \cdot (1\text{-}p)^{2(N-1)}$

… choosing optimum p as n -> infty …

= 1/(2e) = .18



protocol constrains effective channel throughput!

纯ALOHA（续）

给定节点成功发送的概率 = 节点传输的概率 .

P（其他节点在[po-1,p_0] 期间不传输）。

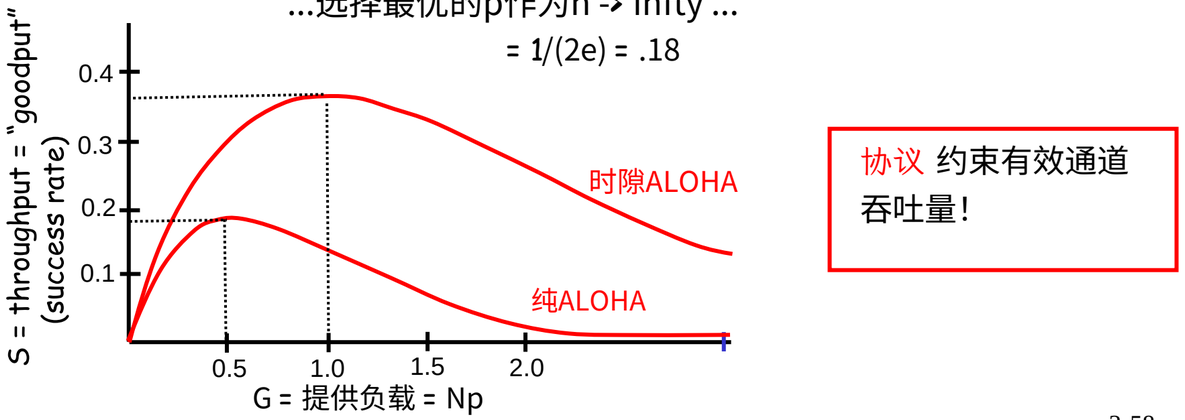
P（其他节点在[po-1,p_0]期间不传输）。

= $p \cdot (1\text{-}p)^{N-1} \cdot (1\text{-}p)^{N-1}$

= $p \cdot (1\text{-}p)^{2(N-1)}$

…选择最优的p作为n -> infty …

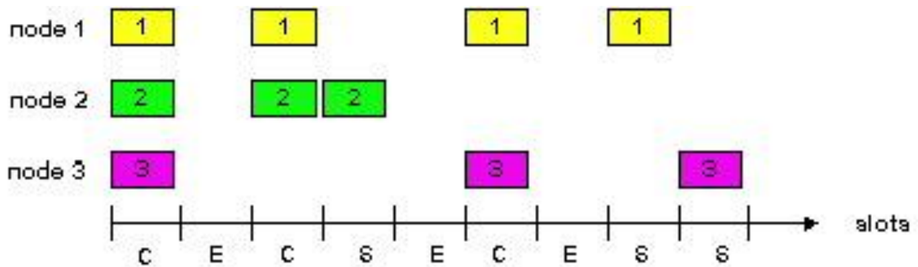
= 1/(2e) = .18



协议 约束有效通道吞吐量!

Slotted Aloha

- time is divided into equal size slots (= pkt trans. time)
- node with new arriving pkt: transmit at beginning of next slot
- if collision: retransmit pkt in future slots with probability p, until successful.

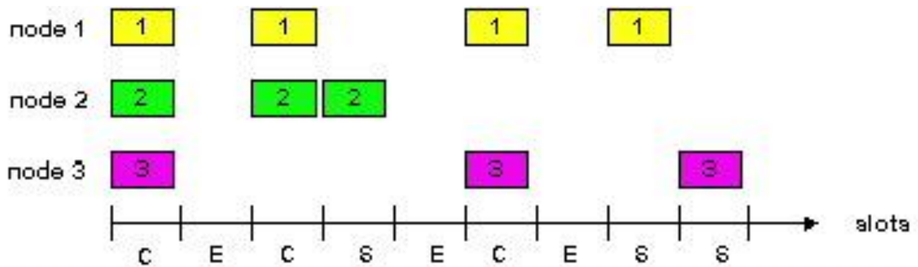


Success (S), Collision (C), Empty (E) slots

时隙ALOHA

- 时间被划分为等大小的槽位（= 数据包传输时间）
- 节点有新到达的数据包：在下一个槽位的开始处传输

- 如果发生冲突：在未来的槽位中以概率 p 重新传输数据包，直到成功。



成功 (S)、冲突 (C)、空闲 (E) 槽位

Slotted Aloha efficiency

Q: what is max fraction slots successful?

A: Suppose N stations have packets to send

- each transmits in slot with probability *p*
- prob. successful transmission *S* is:

by single node: $S = p(1-p)^{(N-1)}$

by any of N nodes

S = Prob (only one transmits)

$= N p (1-p)^{(N-1)}$

... choosing optimum p as n -> infty ...

$= 1/e = .37$ as N -> infty

At best: channel use for useful transmissions 37% of time!

时隙ALOHA效率

Q: 最大成功槽位比例是多少?

A: 假设N个站点有数据包要发送

- 每个站点在槽位中传输的概率为*p*
- 成功传输的概率*S* 为:

单个站点: $S = p(1-p)^{(N-1)}$

任意N个站点S = 只有一个站点传输的概率= N p (1-p)$^{(N-1)}$... 选择最优p作为n -> 无穷大= $1/e$ = .37作为N -> 无穷大

最佳情况: 通道用于有用传输的时间为37%!

CSMA: Carrier Sense Multiple Access)

CSMA: listen before transmit:

- If channel sensed idle: transmit entire frame
- If channel sensed busy, defer transmission
  - Persistent CSMA: retry immediately with probability p when channel becomes idle (may cause instability)
  - Non-persistent CSMA: retry after random interval
- human analogy: don't interrupt others!

CSMA：载波侦听多路访问)

CSMA： 传输前先监听：

- 如果通道空闲：传输整个帧 如果通道繁忙，则延迟传输
  - 持续CSMA： 当通道空闲时立即重试，概率为p（可能导致不稳定）
  - 非持续CSMA： 在随机间隔后重试
- 人类类比：不要打断别人！

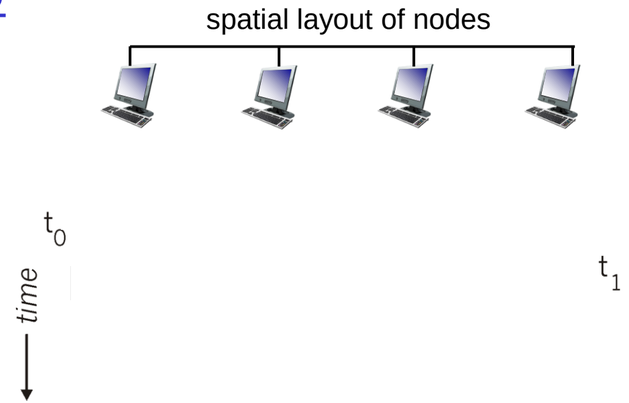
3-61

CSMA collisions

collisions *can* occur:
propagation delay means two nodes may not year hear each other's transmission

collision:
entire packet transmission time wasted

note:
role of distance and propagation delay in determining collision prob.

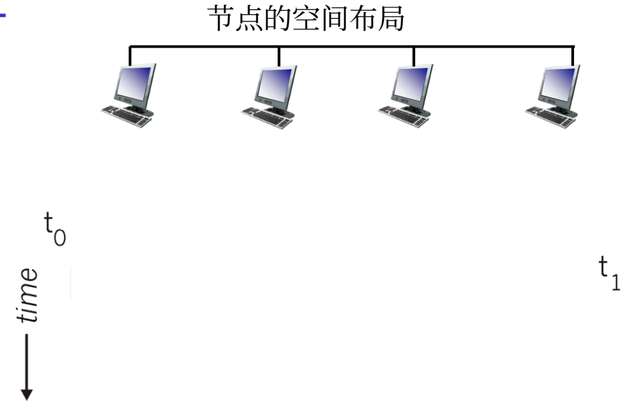


CSMA 冲突

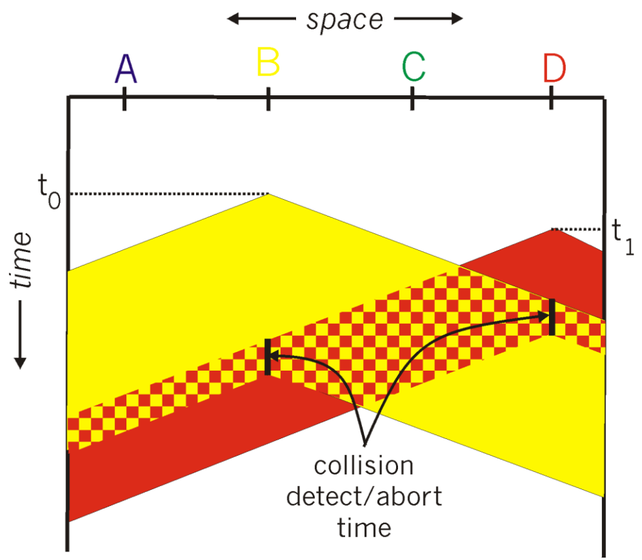
冲突可以发生：传播延迟意味着两个节点可能无法同时听到彼此的传输

冲突：
整个数据包传输时间都浪费了

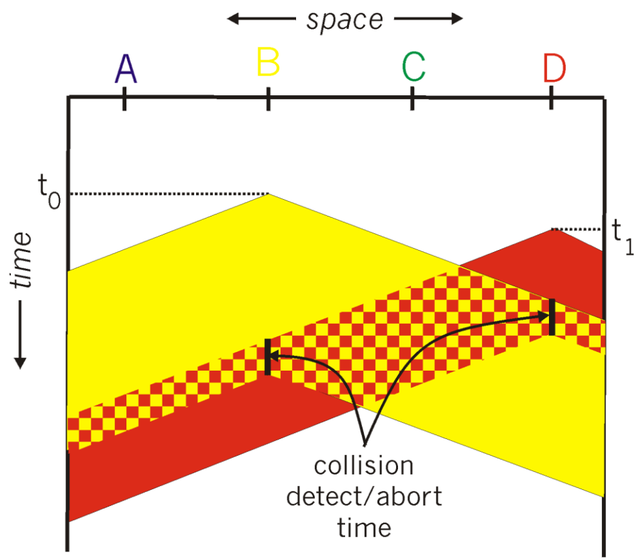
note:
距离和传播延迟在确定冲突概率中的作用



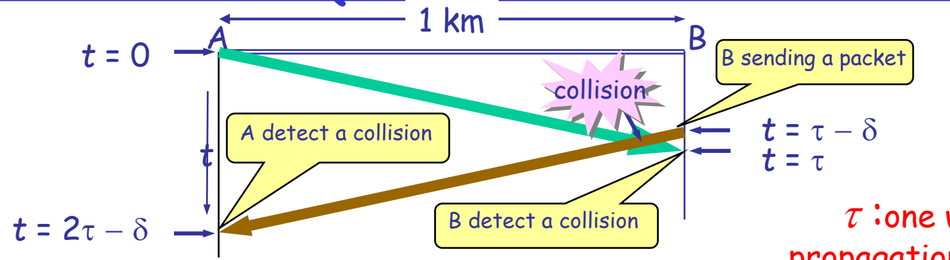
CSMA/CD collision detection



CSMA/CD 冲突检测

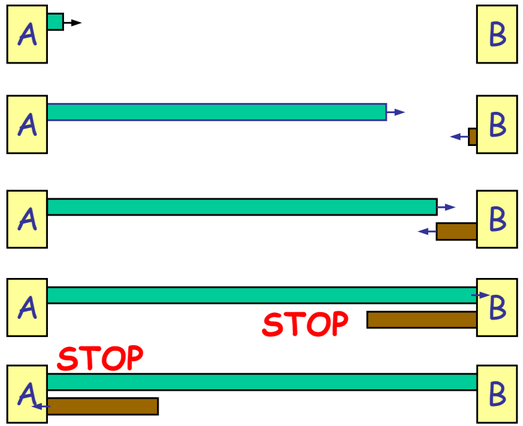


CSMA/CD (Collision Detection)



τ :one way propagation delay

t = 0, A find that the channel is idle, then sending data



t = τ – δ
B find that the channel is idle, then sending data

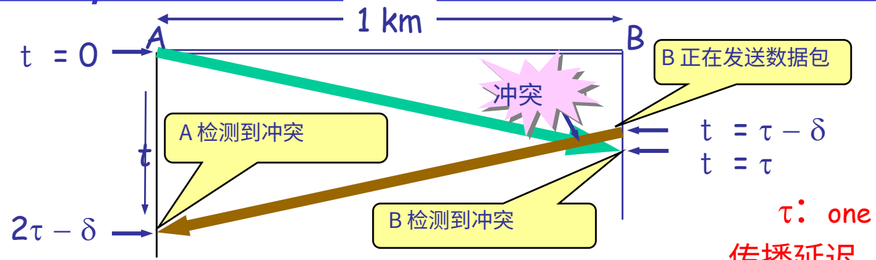
t = τ – δ / 2
collision

t = τ
B detect the collision, then stop sending

t = 2τ – δ
A detect the collision, then stop sending

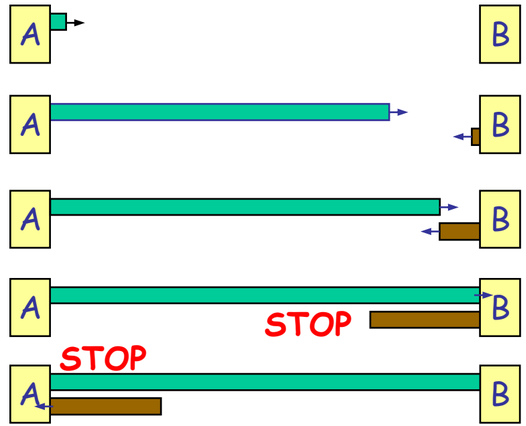
2τ :Contention Period

CSMA/CD（冲突检测）



τ：one 方式传播延迟

t = 0, A 发现 the 通道空闲 然后发送数据



t = τ – δ
B发现通道空闲，然后发送数据

t = τ – δ/ 2
冲突

t = τ
B 检测到冲突 然后停止发送

t = 2τ – δ
A 检测到冲突，然后停止发送

2τ :Contention 周期

CSMA/CD (Collision Detection)

CSMA/CD: carrier sensing, deferral as in CSMA
- collisions *detected* within short time
- colliding transmissions aborted, reducing channel wastage
- persistent or non-persistent retransmission

collision detection:
- easy in wired LANs: measure signal strengths, compare transmitted, received signals
- difficult in wireless LANs: receiver shut off while transmitting

human analogy: the polite conversationalist

CSMA/CD（冲突检测）

CSMA/CD： 载波侦听、退避，与CSMA类似
- 冲突在短时间内被检测到
- 冲突传输被中止，减少通道浪费
- 持续或非持续重传

冲突检测：
- 在有线局域网中容易：测量信号强度，比较传输和接收的信号
- 在无线局域网中困难：接收者在传输时关闭

人类类比：礼貌的交谈者

Summary of MAC protocols

- What do you do with a shared media?
  - Channel Partitioning, by time, frequency or code
    - Time Division,Code Division, Frequency Division
  - Random partitioning (dynamic),
    - ALOHA, S-ALOHA, CSMA, CSMA/CD
    - carrier sensing: easy in some technoligies (wire), hard in others (wireless)
    - CSMA/CD used in Ethernet
  - Taking Turns
    - polling from a central cite, token passing

MAC 协议总结

- 你如何处理共享媒体?
  - 通道划分，按时间、频率或代码
    - 时分、码分、频分
  - 随机划分（动态），
    - ALOHA、S-ALOHA、CSMA、CSMA/CD
    - 载波侦听：某些技术（有线）中容易，在另一些技术（无线）中困难
    - 以太网使用CSMA/CD
  - 轮流
    - 从中央站点轮询，令牌传递

3-66

Summary of MAC protocols

channel partitioning MAC protocols:

- share channel efficiently at high load
- inefficient at low load: delay in channel access, 1/N bandwidth allocated even if only 1 active node!

Random access MAC protocols

- efficient at low load: single node can fully utilize channel
- high load: collision overhead

"taking turns" protocols

look for best of both worlds!

MAC协议总结

通道划分MAC协议：

- 在高负载下高效共享通道
- 在低负载下效率低下：通道访问延迟，即使只有1个活动节点也会分配1/N带宽！

随机接入MAC协议

- 在低负载下高效：单个节点可以完全利用通道
- 在高负载下：冲突开销

"轮流"协议

寻求两全其美！

Exercises-4

- A LAN adopts CSMA/CD protocol to realize the media access control. The data transmission rate is 10Mbps, the distance between host A and host B is 2km, and the signal propagation speed is 200000km/s. If there is a conflict when two hosts send data, how long will it take from the time when they start sending data to the time when both hosts detect the conflict? (What is the minimum and maximum duration?)

习题-4

- 一个局域网采用CSMA/CD协议来实现介质访问控制。数据传输速率为10Mbps，主机A和主机B之间的距离为2km，信号传播速度为200000km/s。如果两台主机发送数据时发生冲突，从它们开始发送数据到两台主机检测到冲突需要多长时间？（（最小和最大持续时间是多少？））

Chapter 3: Summary

- principles behind data link layer services:
  - framing
  - error detection, correction
  - reliable data transfer
  - sharing a broadcast channel: multiple access
- Next, various link layer technologies
  - LAN Model
  - Ethernet
  - WLAN

第三章：摘要

- 数据链路层服务背后的原理：
  - 帧编码、错误检测、纠正、可靠数据传输、共享广播通道：多路访问
- 接下来，各种链路层技术
  - 局域网模型
  - 以太网、无线局域网